OntoDB

Database Internals by Example

Storage, indexing, query processing, concurrency, and recovery,
taught through a database management system for ontologies

Lam Nguyen

Contents

Design Decisions vi

Preface ix

I Foundations 1

1 What a Database Engine Is 2

1.1 The layers . . . 2
1.2 RDF in one page . . . 3
1.3 SPARQL in one page . . . 3
1.4 The running example . . . 4

II Storage 6

2 Disks, Files, and Pages 7

2.1 The storage hierarchy . . . 7
2.2 Pages . . . 7
2.3 Slotted pages . . . 8
2.4 Talking to the file system . . . 9

3 The Buffer Pool 13

3.1 Frames, the page table, and pins . . . 13
3.2 Choosing a victim . . . 14
3.3 Steal and force . . . 14

4 B-Trees and B+ Trees 20

4.1 Why a tree of pages . . . 20
4.2 B-tree versus B+ tree . . . 20
4.3 Search . . . 21
4.4 Insertion and splits . . . 21
4.5 Deletion . . . 21
4.6 Bulk loading . . . 22

4.7 Iterators and pins 22

5 Indexing RDF **26**

5.1 Dictionary encoding 26

5.2 A key whose bytes sort correctly 27

5.3 Prefix scans 27

5.4 One ordering is not enough 27

6 The Log-Structured Alternative **33**

6.1 How an LSM tree works 33

6.2 Three amplifications 33

III Query Processing **35**

7 From SPARQL to a Plan **36**

7.1 Parsing and binding 36

7.2 Logical and physical plans 36

7.3 The iterator (Volcano) model 37

7.4 The operators 37

7.5 Sorting more than fits in memory 38

8 Joins and Query Optimization **42**

8.1 Three join algorithms 42

8.2 Cardinality estimation 42

8.3 Searching for a join order 43

8.4 Explain analyze 43

IV Transactions **49**

9 Transactions and Isolation **50**

9.1 ACID 50

9.2 Schedules and serializability 50

9.3 Anomalies 50

9.4 Isolation levels 51

10 Concurrency Control **54**

10.1 Two-phase locking . . . 54
10.2 Locking at several granularities . . . 54
10.3 Deadlock . . . 55
10.4 Phantoms . . . 55
10.5 Latches are not locks . . . 55
10.6 The alternative: multiversion concurrency . . . 56

V Recovery **58**

11 Logging and the Write-Ahead Rule **59**
11.1 The failure model . . . 59
11.2 Undo and redo . . . 59
11.3 The write-ahead rule . . . 60
11.4 Making commits cheap: group commit . . . 60
11.5 When a record is damaged . . . 60

12 ARIES **63**
12.1 The data structures . . . 63
12.2 Checkpoints . . . 63
12.3 Pass 1: Analysis . . . 63
12.4 Pass 2: Redo . . . 64
12.5 Pass 3: Undo . . . 64
12.6 A worked example . . . 64
12.7 Structure changes: nested top actions . . . 65

13 Durability in Practice **67**
13.1 Where it sits in the steal/force table . . . 67
13.2 Logical logging . . . 67
13.3 One statement, one transaction . . . 68
13.4 Recovery . . . 68
13.5 Checkpoints as atomic snapshots . . . 68
13.6 Damaged logs: recovery modes . . . 69
13.7 Verification . . . 69
13.8 Testing a recovery design . . . 69

A A Map from Chapters to OntoDB **72**

B Answers **73**

Bibliography **80**

Design Decisions

Every choice OntoDB makes is written up where the theory behind it is taught, in a box that names the choice, the alternatives it passed over, why, and what it costs. A title marked *planned* is a decision fixed in the plan for code not yet written.

1 One text interface . . . 4
2 Store OWL as triples, with no reasoner . . . 4
3 4 KiB pages . . . 9
4 Page 0 is a real page . . . 9
5 The file's length records allocation . . . 10
6 The file only grows . . . 10
7 Through the page cache; `fsync` at shutdown . . . 10
8 Compact on delete . . . 10
9 A term must fit on one page . . . 11
10 Append to the last page . . . 11
11 Spellings are never deleted . . . 11
12 One I/O thread, first in, first out . . . 15
13 LRU-K with $K = 2$ . . . 15
14 One latch for the whole pool, held across I/O . . . 16
15 The pool's destructor does not flush . . . 16
16 A full pool returns null . . . 16
17 Pages only through guards . . . 16
18 Pin, then latch; unlatch, then unpin . . . 17
19 A latch belongs to a page, not to a frame . . . 17
20 A write guard dirties only if written . . . 17
21 Inspecting a page goes through the pool . . . 18
22 Fixed slots, no key compression . . . 22
23 The key is the record . . . 23
24 Strict deletion . . . 23
25 The invariant checker comes before insert . . . 23
26 Remember the path, not parent pointers . . . 24
27 Split in the middle . . . 24
28 Bulk load packs leaves full . . . 24

29 Small nodes in tests . . . 24
30 Ids start at 1; 0 and the maximum are reserved . . . 28
31 The dictionary is built from the engine's own pages . . . 28
32 A fixed hash function . . . 29
33 Big-endian keys, compared with `memcmp` . . . 29
34 The upper bound saturates instead of failing . . . 29
35 The catalog points at header pages, not roots . . . 30
36 The catalog claims page 0 by writing it . . . 30
37 SPO, POS, and OSP, measured . . . 30
38 Permutations are data, and every write is all-or-nothing . . . 30
39 The shell's indexed backend lives in a temporary file . . . 31
40 Unsupported syntax fails at parse time . . . 38
41 Volcano, one row at a time . . . 38
42 Stub plan nodes before their executors . . . 39
43 `ORDER BY` uses a total order on values . . . 39
44 External sort through the buffer pool . . . 39
45 `DISTINCT` keeps a hash set . . . 40
46 Rows have no order without `ORDER BY` . . . 40
47 Exact counts for single patterns . . . 44
48 Join estimates use distinct values . . . 44
49 The inner side is re-opened with the outer row . . . 44
50 Index nested loop probes the store directly . . . 45
51 Hash join builds on the inner side . . . 45
52 Forcing a join method rewrites the plan . . . 45
53 Greedy, left-deep join ordering first . . . 45
54 `EXPLAIN ANALYZE` wraps every executor . . . 46
55 `auto` picks the operator per join, and is the default . . . 46
56 Every statement is a transaction . . . 51
57 A failed statement takes itself back . . . 52
58 Read committed by default (planned) . . . 52
59 Isolation is specified as executable schedules . . . 52
60 Locking, not multiversioning (planned) . . . 56
61 Detect deadlocks and abort the youngest (planned) . . . 56
62 The phantom mechanism is a written choice (planned) . . . 56

63 Length and CRC-32C in every record 61
64 LSNs are counters, not byte offsets 61
65 A failed `fsync` is fatal 62
66 A blocking checkpoint first (planned) 65
67 Recovery is tested against an oracle 66
68 Terms replay whatever their transaction did 70
69 Point-in-time recovery by default 70

Preface

A database engine is a small number of ideas, applied with care. Data lives in fixed-size pages because disks move blocks. A buffer pool keeps the hot pages in memory because memory is fast and small. A B+ tree turns a key into a page in three or four reads because its fanout is in the hundreds. A lock manager makes concurrent transactions look sequential. A write-ahead log, and an algorithm called ARIES, make a committed transaction survive a crash that happens at any instant.

This book explains those ideas from the ground up, and it explains each one twice: first as theory, then as it appears in a real system. The real system is **OntoDB**, a database management system for ontologies. Its data is RDF, a graph of subject–predicate–object triples, and its query language is SPARQL. Every chapter ends its theory with an *In OntoDB* box that shows what OntoDB does, and design-decision boxes that show what it chose instead, and why.

Why an ontology database

A relational engine teaches the same ideas, so why a triple store? Two reasons. First, a triple store strips the problem to its core: there is one table, with three columns, and every interesting question is a join. That makes indexing and join ordering impossible to wave away. Second, ontologies are where correctness matters most and is checked least: a scientific ontology edited by many hands needs transactions, a log, and recovery just as much as a bank ledger does.

How to read this book

The parts follow the path of a request through the engine, from the bottom up.

Foundations What an engine is made of, and enough RDF and SPARQL to follow the examples.

Storage Pages, the buffer pool, B+ trees, how RDF is indexed, and the log-structured alternative.

Query processing How a SPARQL query becomes a tree of operators, and how joins are chosen.

Transactions What isolation means, and how locking provides it.

Recovery Logging, the write-ahead rule, ARIES, and how OntoDB makes its databases durable today.

The boxes

Four kinds of box recur. A *Key idea* is the one thing to keep from a section. *In OntoDB* shows how the running example applies the theory. A *Design decision* names a choice OntoDB made, the alternatives, why it chose as it did, and what the choice costs; a list of them follows the contents,

and they are the best place to start if you want to know why the engine is built the way it is. *Status* says what runs today. Each chapter ends with questions; the answers are in appendix B.

Code appears only where a picture or a sentence would be less precise: a key layout, a log record, a few lines of pseudocode for an algorithm. The aim is to understand *why*, not to read an implementation.

Status

OntoDB is built feature by feature, and parts of this book describe designs that are specified but not yet implemented. Each chapter's *Status* box says which. At the time of writing these run: the SPARQL shell; the triple key; the buffer pool (disk manager, I/O scheduler, LRU-K, page guards, and inspection); the B+ tree; RDF storage on pages (term heap, catalog, disk dictionary, and the indexed store, which the shell switches to with `\set backend indexed`); the log records and the log manager; every query operator and the optimizer (statistics, join order, the choice of join, and `\explain analyze`); and durable databases (snapshots, a checksummed write-ahead log, recovery, and verification). Bulk loading, concurrency control, and page-level ARIES are designed and being built.

Acknowledgments

The structure of this book owes much to Alex Petrov's *Database Internals*, whose division into storage engines and distributed systems it follows for the first half. The recovery chapters follow Mohan et al.'s ARIES paper closely, because nothing explains ARIES better than ARIES.

Part I

Foundations

Chapter 1

What a Database Engine Is

A database management system (DBMS) accepts declarative requests, such as "find every student and their name", and answers them correctly and quickly, while many clients read and write at once and while machines fail. Each of those words is a subsystem. *Correctly* under concurrency is the job of the transaction manager. *Quickly* is the job of indexes and the query optimizer. *While machines fail* is the job of the log and recovery.

1.1 The layers

Almost every engine, relational or not, is built from the same layers. A request travels down them, and data travels up.

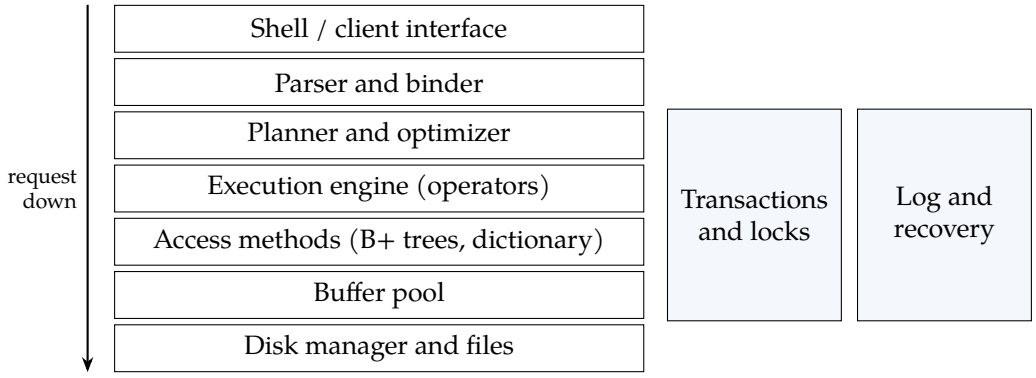


Figure 1.1: The layers of a database engine. Transactions and recovery cut across the stack.

Interface Reads text, returns results. For OntoDB this is the `ontodb` shell.

Parser and binder Turn text into a tree, check names, and replace each constant with an internal id.

Planner and optimizer Choose *how* to answer: which index to use, in what order to join, with which algorithm.

Execution engine Runs the chosen plan as a tree of operators that pass rows to each other.

Access methods The data structures that find data: B+ trees for ranges, hash tables for exact matches, the dictionary for term ids.

Buffer pool Caches disk pages in memory, decides which to evict, and enforces the write-ahead rule when a page goes back to disk.

Disk manager Reads and writes fixed-size pages of a file.

Two subsystems do not fit in one layer. The **transaction manager** and **lock manager** touch every read and write. The **log manager** and **recovery** touch every update, every page write, and every restart. Most of the difficulty of building an engine lives in how these cross-cutting pieces interact with the layers.

Key idea

Each layer hides a cost from the one above. The buffer pool hides the disk; the B+ tree hides how many pages a lookup needs; the optimizer hides that a query can be answered in a thousand ways. Understanding an engine means knowing what each layer hides, and when the hiding fails.

1.2 RDF in one page

RDF represents knowledge as a set of *triples*. A triple is (*subject*, *predicate*, *object*): the subject is a thing, the predicate a relationship, and the object another thing or a value.

```
<http://ex.org/Alice>  <http://ex.org/name>       "Alice"
<http://ex.org/Alice>  rdf:type                   <http://ex.org/Student>
<http://ex.org/Metal>  rdfs:subClassOf            <http://ex.org/Material>
```

Things are named by IRIs (`<http://...>`); values are literals, optionally with a language (`"Métal"@fr`) or a datatype (`"21"^^xsd:integer`). An *ontology* is RDF that describes concepts and their relations: `Metal subClassOf Material`, `Alloy subClassOf Metal`. OWL, the standard ontology language, is itself written as RDF triples, so a triple store can store any OWL ontology without understanding it.

1.3 SPARQL in one page

SPARQL asks questions by *pattern*. A *triple pattern* is a triple in which some positions are variables. A *basic graph pattern* is a set of triple patterns that share variables; its answers are every way of binding the variables so that every pattern becomes a stored triple.

```
PREFIX ub: <http://example.edu/univ#>
SELECT ?name WHERE {
  ?s a ub:Student .        # ?s is a student
  ?s ub:name ?name .       # ... and has a name
}
```

Two patterns that share `?s` are a *join*. That is the single most important fact about evaluating SPARQL: a query with k patterns is a k-way self-join of one large table of triples. Everything in chapter 5 and chapter 8 follows from it.

1.4 The running example

In OntoDB

OntoDB is a database management system for ontologies, written in C++20. It loads Turtle, N-Triples, and RDF/XML (so OWL files load as triples), and it answers SPARQL basic graph patterns with filters, `INSERT DATA`, and `DELETE DATA`. Its layers map one to one onto fig. 1.1: a shell (`tools/shell`), a lexer, parser, and binder (`src/parser`, `src/binder`), a planner and optimizer, volcano executors (`src/execution`), a dictionary and a store (`src/dictionary`, `src/store`), a buffer pool and disk manager (`src/buffer`, `src/storage`), and a log and recovery manager (`src/recovery`).

A single query, followed through OntoDB, touches every layer:

1. The shell reads lines until the query's braces balance.
2. The parser builds a syntax tree; the binder resolves prefixes, turns every IRI and literal into a 64-bit id from the dictionary, and gives each variable a column number.
3. The planner builds a tree: one scan per triple pattern, joins between them, a filter, a projection.
4. The execution engine pulls rows of ids up that tree, one at a time.
5. The shell turns ids back into text through the dictionary and prints them.

An insert adds a sixth step: before the shell says `inserted 1 triple`, a log record describing the insert must be on disk. Why that order is non-negotiable, and how it lets the engine survive a crash, is the subject of part V. First, storage.

Design decision 1: One text interface

Choice. The shell is the only way in. Every caller (the shell, the crash harness, the differential tests) goes through one facade, `Database`, which takes SPARQL text or a backslash command and returns rows. A future query window would send the same text and print the same rows; it would not get its own parser.

Alternatives. A client–server wire protocol, as PostgreSQL and MySQL have; or an embedded library API with one call per operation, as SQLite and RocksDB offer.

Why. One entry point means one code path to test. A crash test, a differential test, and a person at the prompt all exercise exactly the same parser, planner, and log.

Cost. No remote clients. Two "users" are two shell sessions on threads of one process, which is enough to test locking but not a network service.

Design decision 2: Store OWL as triples, with no reasoner

Choice. OWL and RDFS load as plain triples. `Alloy subClassOf Metal` and `Metal subClassOf Material` are two stored facts; the engine does not derive `Alloy subClassOf Material`.

Alternatives. Forward chaining (materialize the closure at load, so queries see derived triples), or backward chaining (rewrite each query to follow `subClassOf` at run time).

Why. The book and the engine are about storage, indexing, transactions, and recovery. Inference is a separate feature with its own costs, and it is on the list of later work, to be measured on LUBM.

Cost. Queries that depend on inference return a subset of the "intended" answer. Several LUBM benchmark queries do exactly that.

Check your understanding

1.1 Name the layer that hides each cost: (a) a lookup needs four page reads; (b) the disk is slow; (c) a query has a thousand possible plans.

1.2 Why does the transaction manager not fit into one layer of fig. 1.1?

1.3 A SPARQL query has five triple patterns that all share `?s`. Viewed as relational algebra over one table of triples, what is it?

1.4 A user loads an ontology with `Alloy subClassOf Metal` and `Metal subClassOf Material` and asks for every subclass of `Material`. What does OntoDB return, and why?

Answers are in appendix B.

Part II

Storage

Chapter 2

Disks, Files, and Pages

Every design decision in a storage engine starts from one fact: memory is fast and volatile, and storage is slow and durable. A database must keep its data on the slow, durable device and still answer queries at the speed of the fast, volatile one. This chapter is about the unit that moves between the two: the *page*.

2.1 The storage hierarchy

Device	Typical access latency	Survives power loss
CPU cache	1–10 ns	no
Main memory (DRAM)	about 100 ns	no
NVMe SSD, random 4 KiB read	10–100 μs	yes
Spinning disk, random read	5–10 ms	yes

Table 2.1: Orders of magnitude, not benchmarks. The gap between memory and storage is a factor of a hundred to a hundred thousand.

Two properties of storage shape everything above it.

Block granularity. A device reads and writes whole blocks (typically 512 bytes or 4 KiB). Reading one byte costs a whole block. So the engine should arrange data so that one block read brings many useful bytes.

Sequential beats random. Even on SSDs, sequential access is cheaper than random access, and on spinning disks it is dramatically cheaper. An engine that writes sequentially (a log) and reads mostly through an index (a tree) uses the device the way it wants to be used.

2.2 Pages

A *page* is the engine's unit of I/O: a fixed-size block of the database file, usually 4, 8, or 16 KiB, aligned to the device's block size. The engine never reads half a page and never writes half a page. Fixing the size has three payoffs:

- **Addressing is arithmetic.** Page i lives at byte offset $i \times$ PAGE_SIZE in the file. A page id is all the engine needs to find it.
- **Memory is interchangeable.** Every buffer-pool frame holds exactly one page, so any frame can hold any page (chapter 3).

- **Failure is bounded.** A write either lands a whole page, or, if power fails mid-write, produces a *torn page*: part old, part new. Because the size is fixed and known, a checksum over the page detects the tear.

Key idea

A page id is a stable name. Pointers inside the database (a B+ tree child, a heap record's location) are page ids, never memory addresses, so they remain valid across restarts and across evictions.

2.3 Slotted pages

Fixed-size records are easy to place: record k starts at byte $k \times$ size. Real data is variable-sized: an IRI might be 20 bytes or 200. The standard answer is the *slotted page*.

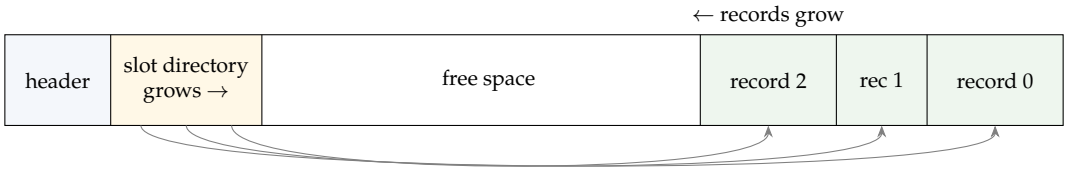


Figure 2.1: A slotted page. The slot directory grows forward from the header, records grow backward from the end, and free space sits between them.

The header records how many slots exist and where free space begins. Each slot holds a record's offset and length. A record is addressed by a *record id* (RID): (*page id, slot number*). Two properties make this work:

- **Records can move without changing their RID.** To compact the page, slide records together and update the slot offsets. The slot number, and therefore the RID, is unchanged.
- **Deletion is cheap.** Mark the slot free; a later insert may reuse it. The space is reclaimed when the page is compacted, or when a new record fits in the hole.

In OntoDB: pages and the term heap

Every OntoDB page is 4096 bytes (`PAGE_SIZE`). The first 8 bytes of *every* page, from the very first version, are reserved for the *pageLSN*, the log sequence number of the last update applied to that page; the payload starts at byte 8. Reserving the field before any logging existed means the file format did not have to change when the write-ahead log arrived (chapter 11).
The spellings of IRIs and literals live in a *term heap* made of slotted pages. The B+ tree leaves never contain strings: they contain 24-byte keys of three ids. That separation keeps the tree's fanout high, and it is why a long IRI costs nothing in the index (chapter 5). A spelling is never deleted when a triple using it is deleted: the dictionary keeps the id, so a log record written earlier can still name it.

2.4 Talking to the file system

An engine reads and writes its file through the operating system, and two details of that conversation decide whether data survives a crash.

The page cache. A `write` call returns as soon as the bytes are in the operating system's page cache, in memory. They reach the device later. A crash after `write` returns can still lose them.

`fsync`. Calling `fsync` on a file asks the operating system to push that file's dirty data to the device and wait until the device reports it durable. It is the only promise of durability the engine gets, and it is expensive: a few milliseconds on a disk, tens to hundreds of microseconds on an SSD. Two consequences follow.

1. **Batch the expensive call.** Engines amortize `fsync` across many transactions; the log manager's *group commit* (chapter 11) exists for this.
2. **Never assume a failed `fsync` can be retried.** On several operating systems, after `fsync` reports an error the kernel may already have discarded the dirty pages and cleared the error, so a second `fsync` that "succeeds" has lost data. PostgreSQL learned this in 2018 and now treats a failed `fsync` as fatal [23, 22].

Atomic replacement. Renaming a file over another is atomic on POSIX file systems: a reader sees the old file or the new one, never a mixture. The idiom "write to a temporary file, `fsync` it, `rename` it over the target, `fsync` the directory" replaces a whole file atomically. OntoDB writes its snapshots this way (chapter 13).

Design decision 3: 4 KiB pages

Choice. `PAGE_SIZE` is 4096 bytes, fixed for the whole system.

Alternatives. 8 KiB (PostgreSQL), 16 KiB (InnoDB), or a size chosen per database at creation.

Why. 4 KiB matches the operating system's page and a common device block, so one page is one block. It is also small enough that tests with a pool of three or four frames, and trees of a few hundred keys, exercise eviction, splits, and merges quickly.

Cost. Lower fanout than a 16 KiB page, so a very large index may need one more level; and fixed per-page overhead (the 8-byte pageLSN, the header) is a larger share of each page.

Design decision 4: Page 0 is a real page

Choice. Page ids are `int32`, and page 0 is a legal page. "No page" is `INVALID_PAGE_ID`, which is -1. The catalog (plan 3.3) will initialize its page explicitly, so a zero-filled file cannot be read as "the root is page 0".

Alternatives. Reserve page 0 as a null page and use 0 as the sentinel.

Why. Zero is the value memory and files have by accident. A sentinel should be a value that never appears unless someone wrote it on purpose; then a missing initialization is a visible error, not a tree rooted at page 0.

Cost. Every reader of a stored page id must check for -1, and the largest file is 2^{31} pages, 8 TiB at 4 KiB.

Design decision 5: The file's length records allocation

Choice. `AllocatePage` grows the database file by one zero page (`ftruncate`, which usually writes no data blocks). On open, the number of allocated pages is the file size divided by `PAGE_SIZE`, rounded up. There is no header page of the disk manager's own.

Alternatives. A header page (a superblock) holding the next page id and a free list, as SQLite's first page and InnoDB's space header do.

Why. Page 0 already belongs to the catalog. The file system updates a file's length as one fact, so there is no second record of allocation that a crash could leave disagreeing with the file.

Cost. The new length is durable only after an `fsync`, so after a crash a page that a log record names may lie past the end of the file. The disk manager therefore reads any page past the end as zeros and grows the file when one is written there, which lets redo apply such a record with no special case.

Design decision 6: The file only grows

Choice. `DeallocatePage` does not return the id for reuse. A freed page stays in the file, unused.

Alternatives. An in-memory free list, or a durable free-space map such as PostgreSQL's.

Why. An id that is reused while anything still names it (a log record awaiting redo, a stale parent pointer after a crash) turns an old reference into a write to someone else's page. It is the same argument as for term ids: an id's meaning should not change.

Cost. Space freed by merges and deletes is not reclaimed. Reclaiming it safely needs a logged free-space map, which is later work.

Design decision 7: Through the page cache; `fsync` at shutdown

Choice. Pages are read and written with `pread` and `pwrite` through the operating system's page cache. `WritePage` does not `fsync`; `ShutDown` does, once.

Alternatives. `O_DIRECT`, bypassing the page cache as InnoDB can; or `fsync` after every page write.

Why. Durability is the log's job: a committed change survives because its log record is forced (chapter 11), not because its page is. A page needs to be on disk only at a checkpoint or a clean shutdown. `pread` and `pwrite` carry their own offset, so threads share one file descriptor without a lock around a seek.

Cost. A cached page exists twice, in a pool frame and in the kernel's cache. Until page-level logging exists (plan 8.3), only a clean shutdown makes the page store durable.

Design decision 8: Compact on delete

Choice. Deleting a record from a term-heap page slides every record stored below it up by the record's length and updates their slot offsets, so the page's free space is always one gap. Slot numbers, and so RIDs, do not change.

Alternatives. Mark the slot free and leave a hole, compacting only when an insert cannot find room, as PostgreSQL defragments a page lazily.

Why. Free space is then exactly one number, an insert never has to search for a hole that fits, and the page code stays small.

Cost. A delete moves up to a page of bytes. Terms are rarely deleted (the dictionary never deletes them), so this cost is almost never paid.

Design decision 9: A term must fit on one page

Choice. A record is at most $4096 - 16 - 4 = 4076$ bytes. A longer spelling is refused with an error, and the load or statement that carried it fails.

Alternatives. Overflow pages chained from the record, or storing large values out of line as PostgreSQL's TOAST does.

Why. IRIs and almost all literals are far shorter, and a record that lives on one page is read with one pin and never reassembled.

Cost. A long `rdfs:comment` or an embedded document cannot be stored. Overflow records are the natural next step if real ontologies need them.

Design decision 10: Append to the last page

Choice. The heap is a chain of pages. An insert goes to the last page, and a new page is chained on when the record does not fit there.

Alternatives. A free-space map that remembers how much room each page has, so an insert can use space freed anywhere.

Why. Terms are added and almost never removed, so the last page is nearly always the only one with room. Appending keeps new terms together.

Cost. Space freed on an earlier page is reused only by records that are placed on that page later, which does not happen; it is lost until a compaction of the whole heap.

Design decision 11: Spellings are never deleted

Choice. Deleting the last triple that uses a term keeps the term and its id in the dictionary.

Alternatives. Reference-count terms and reclaim an id when its count reaches zero; or garbage-collect unused terms at checkpoint.

Why. Log records name terms by id. If id 18 could be reclaimed and reused, an old record that says "insert $(18, 9, 4)$" would silently mean a different triple on replay. Never deleting makes an id's meaning permanent.

Cost. The dictionary only grows. A workload that churns through unique literals keeps every one of them.

Status

The page frame and the pageLSN layout exist, and the disk manager runs (plan 1.1): it allocates pages by growing the file, reads and writes whole pages, and fsyncs on shutdown. The term heap runs too (plan 3.1): slotted pages that compact on delete, chained into a heap that appends to its last page.

Check your understanding

2.1 Page 37 of a 4 KiB-page file: at what byte offset does it start, and why does the engine never need a table to find it?

2.2 In a slotted page, a record is moved during compaction. Why does no index entry that points to it need to change?

2.3 A `write` returned successfully and then the machine lost power. Was the data durable? What call would have made it so?

2.4 `fsync` returns an error. Why is calling it again and continuing on success wrong?

2.5 Why is -1 a better "no page" sentinel than 0?

2.6 A 5,000-byte `rdfs:comment` literal arrives in a file being loaded. What does OntoDB do, and how would PostgreSQL store it?

2.7 After a crash, a log record names page 812, but the file holds only 800 pages. How can OntoDB's allocation scheme produce this, and who must repair it?

Answers are in appendix B.

Further reading

Petrov [1], chapters 2 and 3, on file formats and slotted pages. Pillai et al. [22] on what file systems actually guarantee.

Chapter 3

The Buffer Pool

The database is larger than memory, but queries must run at memory speed. The *buffer pool* bridges the two: a fixed array of in-memory *frames*, each able to hold one page, plus the bookkeeping that decides which pages are in memory and when they go back to disk.

3.1 Frames, the page table, and pins

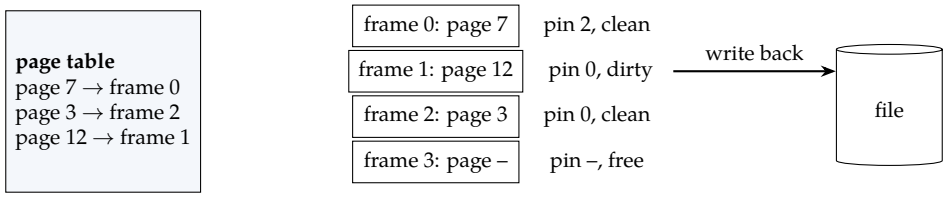


Figure 3.1: A buffer pool with four frames. The page table maps page ids to frames; each frame has a pin count and a dirty bit.

Each frame carries two pieces of state:

Pin count. How many threads are using the page right now. A pinned page must not be evicted: someone holds a pointer into it.

Dirty bit. Whether the in-memory copy differs from the one on disk. A dirty page must be written before its frame is reused.

The interface follows from that:

- `FetchPage(id)`: if the page is in a frame, increment its pin and return it (a *hit*); otherwise choose a victim frame, write it back if dirty, read the page into it, pin it (a *miss*).
- `UnpinPage(id, dirty)`: decrement the pin; mark the frame dirty if the caller changed it.
- `NewPage()`: allocate a page id on disk and return a zeroed, pinned frame.
- `FlushPage(id)`: write a dirty page now.

Key idea

A pin is a promise, and a leaked pin is a leak of memory the pool can never reclaim. Engines therefore wrap pins in a scoped object (a *page guard*) that unpins exactly once when it goes out of scope.

3.2 Choosing a victim

When every frame is occupied and a miss arrives, the pool must evict an unpinned page. The choice matters: a bad policy turns hits into misses, and every miss is a disk read.

LRU. Evict the least recently used page. Simple, and good for many workloads. Its weakness is *sequential flooding*: one large scan touches every page once, and each touch makes a cold scan page look "recent", so the scan flushes out the hot index pages that every other query needs.

CLOCK. An approximation of LRU that avoids moving pages in a list on every access: each frame has a reference bit set on access, and a "clock hand" sweeps the frames, clearing bits and evicting the first frame whose bit is already clear. Cheap, and still vulnerable to flooding.

LRU-K. O'Neil, O'Neil, and Weikum's fix for flooding [9]. Remember the last K access times of each page. The *backward K-distance* of a page is the time since its K-th most recent access; a page accessed fewer than K times has infinite distance. Evict the page with the largest distance.

With $K = 2$, a page touched once by a scan has infinite distance, while an index root touched thousands of times has a small one. The scan page goes first. LRU is the special case $K = 1$.

Page	Accesses (time)	2nd most recent	Backward 2-distance at $t = 10$
A (index root)	1, 4, 8, 9	8	2
B (index leaf)	3, 6	3	7
C (scanned once)	7	—	∞

Table 3.1: LRU-K with $K = 2$. LRU would evict B (least recently used at time 6); LRU-K evicts C, the page with no second access, even though C was touched more recently than B.

When several pages have infinite distance, a tie-breaking rule is needed. A common choice is the page whose oldest remembered access is earliest; with $K = 2$ such a page has exactly one access, so this is plain LRU among the pages seen once.

3.3 Steal and force

Two policy questions connect the buffer pool to recovery, and their answers decide what the log must contain.

Steal? May the pool write a dirty page of an *uncommitted* transaction to disk, to free its frame? If yes, the disk can hold changes that must later be *undone*.

Force? Must every page a transaction changed be written to disk *before* it commits? If no, a committed change may exist only in memory at the time of a crash, and must later be *redone*.

Steal lets the pool evict any unpinned page, so a long transaction cannot exhaust memory. No force lets a commit write one small log record instead of every page it touched. Both are wins,

	Force	No force
No steal	no undo, no redo (slow, memory-bound)	redo only
Steal	undo only	undo and redo (fast, flexible)

Table 3.2: The four combinations. High-performance engines choose steal and no force, and pay for it with a log that supports both undo and redo.

and chapters 11 and 12 show how the log makes them safe.

In OntoDB: the buffer pool

OntoDB's pool uses **LRU-K** with $K = 2$ by default (`\set lru_k`), tie-breaking by the older K-th access. Its policy is **steal / no force**. Pages are reached only through `ReadPageGuard` (shared latch, one pin) and `WritePageGuard` (exclusive latch, one pin); both unpin exactly once. The test harness asserts that the number of pinned frames is zero after every query and every destroyed iterator: a leaked pin fails the test even when the query's rows were right.

When the pool writes a dirty page and a log manager is installed, the order is fixed: first flush the log up to the page's pageLSN, then write the page. A named crash point sits between the two, so a test can kill the process after the log is durable and before the page is. That ordering is the write-ahead rule of chapter 11.

Design decision 12: One I/O thread, first in, first out

Choice. The pool never calls the disk manager itself. It hands a request to a `DiskScheduler`, whose single background thread runs reads and writes in the order they were scheduled. `Schedule` returns at once with a future; if the I/O fails, the error is rethrown from the future's `get`.

Alternatives. Synchronous I/O on the query thread; a pool of I/O threads; an asynchronous kernel interface such as `io_uring`.

Why. In-order execution makes "write page 9, then read page 9" correct with no per-page bookkeeping. A background thread lets a flush or a prefetch overlap with query work, and errors arrive where the waiting caller can handle them.

Cost. No parallel I/O: an SSD reaches full throughput only with many requests in flight, and one slow read delays every request behind it.

Design decision 13: LRU-K with $K = 2$

Choice. Evict the unpinned frame with the largest backward 2-distance. Frames seen fewer than twice have infinite distance and go first, earliest oldest access first. Time is a logical clock that ticks on every access, not the wall clock, and a pinned frame keeps its history.

Alternatives. LRU, CLOCK, 2Q, ARC.

Why. SPARQL plans scan: an unselective pattern reads every leaf of a permutation once. LRU and CLOCK let that scan push out the index's upper levels; LRU-K does not. A logical

clock and a fixed tie-break make eviction deterministic, so a test can predict exactly which frame goes; the replacer's tests replay table 3.1 and check that C, not B, is evicted. Keeping history while a frame is pinned stops a hot page from looking new each time it is unpinned.

Cost. Each frame keeps K timestamps, and choosing a victim scans every frame: linear in the pool size, which is cheap for the default 64 frames but would need a priority structure for pools of millions. A second policy is planned as a measured comparison, not as a replacement.

Design decision 14: One latch for the whole pool, held across I/O

Choice. A single mutex covers the page table, the free list, every pin count and dirty bit, and the replacer calls. It stays held while a miss reads its page or a victim is written back.

Alternatives. Partition the page table and give each frame an "I/O in progress" state, so other threads keep working during a read; this is roughly what PostgreSQL does with buffer-mapping partitions and per-buffer I/O locks.

Why. The pool's state is then always consistent: no frame is half-loaded while another thread looks it up, and the tests pass under ThreadSanitizer without subtle protocols. With a single I/O thread (design decision on the scheduler), finer locking would mostly move the waiting elsewhere.

Cost. A miss stalls every thread for one disk read. Finer-grained latching is the first thing to change when I/O becomes parallel.

Design decision 15: The pool's destructor does not flush

Choice. Destroying the pool discards dirty frames. Durable shutdown calls `FlushAllPages` explicitly (plan 3.6).

Alternatives. Flush every dirty page on destruction, which is convenient.

Why. Durability is the log's job. Tests need a path that behaves like a dead process, writing nothing, and a destructor also runs while an exception from a crash point unwinds; a flushing destructor would write exactly the page the test is trying to keep off disk.

Cost. A caller who forgets `FlushAllPages` loses work on a clean exit. The clean-restart test of plan 3.6 is there to catch it.

Design decision 16: A full pool returns null

Choice. When every frame is pinned, `NewPage` and `FetchPage` return `nullptr` at once.

Alternatives. Block until some frame is unpinned, or throw.

Why. The caller usually holds pins of its own; blocking could wait for a pin that only the waiter can release. Returning lets the caller give up the operation and release what it holds.

Cost. Every caller must check for null; guards and the tree turn that into an error close to the cause.

Design decision 17: Pages only through guards

Choice. Code above the pool reaches a page only through a `ReadPageGuard` or `WritePageGuard`: one pin plus a latch, released exactly once by `Drop` or the destructor. Guards move but never copy, and a moved-from guard is empty. Tests assert the pinned-frame count is zero after every query.

Alternatives. Callers call `UnpinPage` by hand, as early textbook pools do.

Why. A leaked pin returns the right rows and silently shrinks the pool; a double unpin lets a page in use be evicted. Neither is visible in query results. Tying the pin to a scope removes both.

Cost. Ownership rules (move-only, drop before pinning the sibling) that every caller must follow, and a little ceremony for one-line accesses.

Design decision 18: Pin, then latch; unlatch, then unpin

Choice. Each frame has a reader–writer latch. A guard asks the pool to pin the page, then takes the latch, outside the pool's own latch. Dropping releases the latch first and the pin second. A null page from the pool becomes an error at the call that asked for it.

Alternatives. Take the page latch while still holding the pool latch; or release the pin before the latch.

Why. Waiting for a page latch while holding the pool latch would block every thread behind one busy page, and could deadlock with the thread that holds the page and needs the pool. Unpinning first would let the frame be evicted and reused while its latch is still held.

Cost. Between the pin and the latch, another writer may get the page first; callers must not assume what they will find.

Design decision 19: A latch belongs to a page, not to a frame

Choice. When a frame is given to a different page, it gets a new latch. This happens only while the frame is unpinned, and since guards pin before they latch, no thread holds or waits on the old one.

Alternatives. One latch per frame for the life of the pool.

Why. Latch order is a rule about pages: top-down and left to right. With a latch per frame, two unrelated pages that once shared a frame look like the same lock, and ThreadSanitizer reported a lock-order cycle in a bulk load on a three-frame pool that could never deadlock. A fresh latch makes the checker see the same thing the protocol is about.

Cost. A small allocation on every page load, next to a disk read that costs far more. (It has to be a new allocation: ThreadSanitizer forgets a mutex when its memory is freed, but not when a statically initialized `std::shared_mutex` is destroyed and rebuilt in place, which was the first attempt.)

Design decision 20: A write guard dirties only if written

Choice. A write guard marks the page dirty on drop only if its mutable `GetData` was called.

Alternatives. Every write guard marks its page dirty.

Why. B+ tree latch crabbing takes exclusive latches on pages it may never change. Marking those dirty would write clean pages back to disk.

Cost. Code that writes through a pointer obtained earlier, after dropping the guard, escapes the dirty bit, which is one more reason never to keep that pointer.

Design decision 21: Inspecting a page goes through the pool

Choice. `\page N` reads page N with a read guard, as any operator would, and prints its pageLSN and the first payload bytes. `\bpm` prints each frame's page, pins, dirty bit, and pageLSN.

Alternatives. Read the page straight from the file, bypassing the pool.

Why. The pool's copy is what the engine sees, including changes that are dirty and not yet on disk. Reading the file would show a page the engine has already moved past.

Cost. Looking is an access: it can load the page, evict another, and add to the page's LRU-K history. The observer changes what it observes, so `\bpm` is best read before `\page`.

Status

The disk manager, the I/O scheduler, the LRU-K replacer, and the pool run (plans 1.1 to 1.4), and so do page guards with per-frame reader–writer latches (plan 1.5) and the shell's `\bpm`, `\page`, and `\trace` (plan 1.6): the buffer pool is complete. It already flushes the log up to a page's pageLSN before writing the page. The default memory backend has no pool, so these commands come alive with the indexed store (plan 3.4). The experiment that checks them is simple: run a scan with a pool of 4 frames and of 64, with $K = 1$ and $K = 2$, and compare hits and misses in `\stats`.

Check your understanding

3.1 A frame has pin count 0 and is dirty. What must happen before a different page is read into it?

3.2 In table 3.1, add an access to C at time 10. Which page does LRU-K evict at time 11 now?

3.3 With $K = 3$, frame X has accesses at times 2 and 9, and frame Y only at time 5. Both are unpinned. Which does OntoDB evict, and why is X's more recent access irrelevant?

3.4 Explain sequential flooding, and why $K = 2$ resists it while $K = 1$ does not.

3.5 Which cell of the steal/force table needs *undo* information in the log, and why?

3.6 A thread schedules a write of page 9 and then, without waiting, a read of page 9 into another buffer. What does the read return in OntoDB, and what could go wrong with a pool of I/O threads?

3.7 Two threads miss on two different pages at the same moment. In OntoDB's pool, how long does the second thread wait, and why?

3.8 A test arms the crash point between the log flush and the page write, and the exception unwinds through the pool's destructor. Why would a destructor that flushes dirty pages ruin the test?

3.9 Why does a guard release its latch *before* it unpins the page, and not after?

3.10 You run `\page 7` twice and then `\bpm`. How has inspecting the page changed the pool?

3.11 A query returns the right rows, but afterwards one frame is still pinned. What will go wrong later, and how do OntoDB's guards prevent it?

Answers are in appendix B.

Further reading

Petrov [1], chapter 5 on buffer management. O'Neil et al. [9]. Effelsberg and Härder [10] on buffer management principles.

Chapter 4

B-Trees and B+ Trees

An index answers "where is the data with this key?" without reading all the data. For data on disk the answer has been the same since 1970: the B-tree of Bayer and McCreight [5], and its variant the B+ tree, described by Comer as ubiquitous [6]. This chapter explains why it fits disks so well, and how it stays balanced.

4.1 Why a tree of pages

A binary search tree has fanout 2: each node has two children, so finding one of N keys takes about $\log_2 N$ steps. If every step is a page read, a billion keys need 30 reads. A B-tree makes each node a whole *page* and packs as many keys into it as fit. With fanout F, a lookup takes $\log_F N$ reads.

Fanout F	Height for 10^9 keys	Height for 10^{12} keys
2	30	40
100	5	6
500	4	5

Table 4.1: Height is $\lceil \log_F N \rceil$. A large fanout turns a billion-key lookup into four page reads, and the top levels are almost always in the buffer pool.

The upper levels are tiny (a tree of fanout 500 has 1 root and 500 children above a quarter-million leaves) and hot, so they stay in the buffer pool. In practice a lookup costs one or two actual disk reads.

Key idea

A B-tree is a binary search tree redesigned for block devices: make each node a page, make the fanout as large as the page allows, and the tree is shallow enough that its top fits in memory.

4.2 B-tree versus B+ tree

A *B-tree* stores keys and values in every node. A *B+ tree* stores values only in the *leaves*; internal nodes hold only *separator keys* that direct the search. Two consequences favor the B+ tree for databases:

- **Higher fanout.** Internal nodes hold no values, so more separators fit per page and the tree is shallower.

- **Fast range scans.** Leaves are linked to their right sibling. A range query finds its first key with one descent, then walks sideways along the leaf chain without climbing back up.

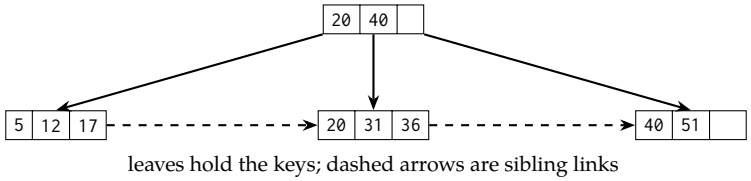


Figure 4.1: A B+ tree. Keys below separator 20 are in the left child; keys from 20 up to 40 in the middle; 40 and above in the right.

The *separator rule* is the tree's central invariant: in an internal node with separators $k_1 < k_2 < \cdots < k_n$ and children $c_0, \ldots, c_n$, every key in c_{i-1} is less than k_i, and every key in c_i is at least k_i.

4.3 Search

Start at the root. At each internal node, binary-search the separators to pick the child whose range contains the key, and descend. At the leaf, binary-search for the key. A range scan $[\ell, u)$ searches for ℓ, then follows sibling links until a key reaches u.

4.4 Insertion and splits

Insert into the leaf the search reaches. If the leaf has room, done. If it is full, *split* it:

1. Allocate a new leaf. Move the upper half of the keys into it.
2. Link it: the new leaf's sibling is the old leaf's sibling, and the old leaf's sibling becomes the new leaf.
3. Insert a separator into the parent: the first key of the new leaf, which is *copied up* (it stays in the leaf).
4. If the parent is now full, split it too. An internal split *pushes up* its middle key: the key moves to the grandparent and is removed from both halves.
5. If the root splits, create a new root with two children. This is the only way the tree grows taller, so every leaf is always at the same depth.

A classic bug lives in step 3: if the middle key is copied up *and* left in the *left* leaf, the separator rule breaks, because the right child's keys must be at least the separator while a copy of it sits on the left. Writing an invariant checker before writing insert catches this at once.

4.5 Deletion

Remove the key from its leaf. If the leaf is still at least half full, done. Otherwise:

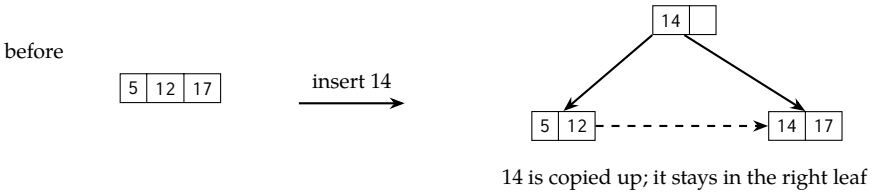


Figure 4.2: A full leaf with capacity 3 splits on insert. The new right leaf's first key becomes the separator.

- **Redistribute**: if a sibling has more than half, move a key across and update the parent's separator, which must now reflect the new boundary.
- **Merge**: otherwise combine the leaf with a sibling, remove the separator between them from the parent, and free the empty page. The parent may now underflow, and the process repeats upward; if the root is left with one child, that child becomes the root.

Many production systems relax deletion and allow underfull nodes, merging only when a node empties: deletes are rarer than inserts, and a tree that gets a little sparse rarely matters.

4.6 Bulk loading

Building a tree by inserting keys one at a time leaves leaves about two thirds full and touches pages in random order. When the keys are available up front, *bulk loading* is far better: sort the keys, fill leaves completely from left to right, link them, then build each internal level from the first keys of the level below. One sequential pass per level, and every page is full.

4.7 Iterators and pins

A range scan returns rows one at a time to the query engine, so it is an *iterator* that holds a position: the current leaf and an index inside it. The leaf must stay pinned while the iterator points into it. When the iterator moves to the sibling, it should unpin the current leaf *before* pinning the next one; holding both, plus a parent, can exhaust a small buffer pool and deadlock.

In OntoDB: the B+ tree

OntoDB's index is a B+ tree. Its key is a 24-byte `TripleKey` (chapter 5), and the leaf stores the key itself: the triple *is* the record, so there is no separate tuple id. Leaves link to their right sibling, the header page stores the root (or `INVALID_PAGE_ID` for an empty tree), and a scan holds at most one pinned leaf at a time.
An invariant checker (`CheckInvariants`) verifies sizes, key order, sibling links, and the separator rule, and the shell can draw the tree (`\tree spo dot`). Splits and merges are multi-page operations, so they have named crash points where a recovery test may kill the process; chapter 12 explains why a split is redone but never undone. Bulk load sorts ids, not strings, because the tree's order is the order of the ids.

Design decision 22: Fixed slots, no key compression

Choice. A page header of 12 bytes follows the pageLSN: type, size, and max size. A leaf then holds its right sibling's id and an array of 24-byte keys, 169 per 4 KiB page. An internal page holds slots of a 24-byte key and a 4-byte child id, 144 keys and 145 children per page. Every key is stored whole.

Alternatives. Prefix compression, which stores each key as the bytes it does not share with its neighbour (most keys in an SPO leaf share their subject), or a slotted layout with variable-length keys.

Why. Fixed slots make `KeyAt(i)` an address computation and binary search trivial, and a key copied out of a page is complete on its own. With fanouts above 140, a billion triples still fit in a tree of five levels.

Cost. Keys that share a long prefix waste most of their bytes, roughly a third of each leaf for a subject-heavy SPO index. Leaf compression is on the list of later work, with the question of what it does to prefix bounds.

Design decision 23: The key is the record

Choice. A leaf stores the 24-byte `TripleKey` and nothing else. There is no value field and no record id pointing to a heap.

Alternatives. The relational layout: a heap file of tuples, with each index mapping a key to a record id.

Why. A triple of ids *is* its whole content. A heap would store the same 24 bytes again and cost an extra page read per match.

Cost. Each permutation is a complete copy of the data, so the number of permutations is a space decision (chapter 5).

Design decision 24: Strict deletion

Choice. `Remove` keeps every node except the root at least half full, by redistributing from a sibling or merging with it.

Alternatives. Lazy deletion, as many production systems do: let nodes become sparse and free a page only when it is empty.

Why. The invariant checker can then assert a fill factor, and merges become real multi-page structure changes with crash points, which the recovery chapters need.

Cost. More page writes on delete-heavy workloads, and the most intricate code in the tree.

Design decision 25: The invariant checker comes before insert

Choice. `CheckInvariants` (size, order, sibling links, the separator rule) is built and tested before `Insert`, and every insert and delete test runs it.

Alternatives. Test insert by looking up the keys afterwards.

Why. Lookups can succeed on a tree that is already wrong; the copied-up-key bug of this chapter does exactly that. A checker turns a structural mistake into a failure on the operation that caused it.

Cost. Code that never runs in production paths, and a full-tree walk in every test.

Design decision 26: Remember the path, not parent pointers

Choice. Pages store no pointer to their parent. An insert or delete records the page ids on its way down, releasing each page before reading its child, and re-fetches a parent by id only if a split or merge reaches it. A descent pins one page at a time; a merge pins three (parent, node, sibling).

Alternatives. A parent pointer in every page; or keeping every page on the path pinned and latched until the operation ends.

Why. A split of an internal page with 145 children would have to rewrite the parent pointer of half of them, which is dozens of page writes for one insert. Releasing pages on the way down lets the whole tree run on a pool of three frames.

Cost. It is correct only while one thread changes the tree: another writer could split a remembered parent between the descent and the re-fetch. Latch crabbing (plan 7.1) replaces this with holding exactly the pages a structure change can reach.

Design decision 27: Split in the middle

Choice. A full leaf splits into two halves, and the right half's first key is copied up. A full internal page splits around its middle key, which moves up.

Alternatives. Split at the insertion point when keys arrive in increasing order, leaving the left page full, as several engines do for append-heavy indexes.

Why. Triples arrive in no particular order across three permutations, and a middle split is the simplest rule that keeps every page at least half full.

Cost. Inserting sorted keys one at a time leaves every leaf half full, twice the pages a scan must read. Loading sorted data is what bulk load is for.

Design decision 28: Bulk load packs leaves full

Choice. `BulkLoad` fills each leaf to its maximum, left to right, then builds each internal level the same way. If the last node of a level would fall below its minimum, it and its left neighbour split their entries evenly.

Alternatives. Leave free space in each leaf, a *fill factor*, as PostgreSQL does for its B-tree indexes (90% by default).

Why. An ontology is loaded and then queried. Full leaves mean the fewest pages for every scan, and loading is one sequential pass per level.

Cost. The first insert into a full leaf splits it at once, so a tree that is loaded and then edited heavily ends up looking like one built by inserts.

Design decision 29: Small nodes in tests

Choice. The tree's maximum node sizes default to what a page holds (169 and 144 keys), and tests pass sizes as small as 2 to grow deep trees from a few dozen keys. A randomized test inserts and deletes against a reference set with node sizes of 2 to 5, checking every invariant after every operation.

Alternatives. Test only with full pages, or shrink `PAGE_SIZE` for tests.

Why. With 169 keys per leaf, a tree needs tens of thousands of keys to reach three levels, so bugs in internal merges and root changes would hide. The same code runs with either size; only two numbers change.

Cost. Small nodes make some cases more common than they are in real use and others rarer, so the full-size tests stay as well.

Status

The B+ tree is complete (plans 2.1 to 2.7): the page layout, the invariant checker and `ToDot`, point search, insert with splits, the prefix iterator, bulk load, and delete with merge and redistribution, with all four crash points. It is single-threaded until latch crabbing (plan 7.1), and the shell's `\tree` draws it once the indexed backend exists (plan 3.4).

Check your understanding

4.1 OntoDB's internal page has 145 slots of 28 bytes after a 20-byte header. Where do the 28 bytes come from, and why does slot 0's key go unused?

4.2 A page holds 300 separators. About how many levels does a tree of 10^8 keys need, and how many of those are likely in the buffer pool?

4.3 During a leaf split the separator 14 is copied up. Which leaf must it stay in, and what breaks if it stays in the other one?

4.4 Why does an internal split *push up* its middle key, while a leaf split *copies* it?

4.5 Why should a scan unpin its current leaf before pinning the sibling?

4.6 Why does OntoDB not store a parent pointer in each page? What does the choice cost once several threads change the tree?

4.7 OntoDB bulk-loads 170 sorted keys with a leaf maximum of 169. What leaves does it build, and why not one leaf of 169 and one of 1?

4.8 Bulk load fills every leaf. Why can't a loop of `Insert` calls do the same?

Answers are in appendix B.

Further reading

Bayer and McCreight [5]; Comer [6]; Petrov [1], chapters 2, 4, and 6.

Chapter 5

Indexing RDF

A relational table has a schema, and each column can be indexed on its own. An RDF store has one table with three columns, and a query can bind any subset of them. This chapter shows how a triple store turns that into fixed-size keys in B+ trees: first by replacing strings with integers, then by laying out the integers so that byte order is query order, and finally by keeping more than one ordering of the same triples.

5.1 Dictionary encoding

IRIs are long and repeated: the same predicate appears in millions of triples. Storing strings in the index would waste space and make every comparison a string comparison. Instead, a *dictionary* maps each distinct term to a dense integer id, once.

id	canonical spelling
1	`<http://example.edu/univ#Alice>`
2	`<http://www.w3.org/1999/02/22-rdf-syntax-ns#type>`
3	`<http://example.edu/univ#Student>`
4	`"Alice"`

A triple becomes three integers, `(1, 2, 3)`. Comparing ids is one machine instruction, and the index stores 24 bytes per triple regardless of how long the IRIs are.

Two rules keep the dictionary correct:

- **One spelling per term.** The loader and the query binder must produce the same canonical text for the same term, or one IRI becomes two ids and every join on it silently fails. Escapes, datatype IRIs, and language tags are part of the spelling.
- **Ids never change.** An id assigned once keeps its meaning across restarts. A log record that names id 18 must still mean the same term when it is replayed (chapter 13).

In OntoDB: the dictionary

Ids are 64-bit and start at 1; 0 is never a valid id, so a zeroed key never names a real term, and the maximum value means "unbound" and is never assigned (see the design decision below). A single codec produces canonical spellings for both the loader and the binder: `<iri>`, `"lex"`, `"lex"@en`, `"lex"^^<datatype>`, and `_:b0`. Numeric literals compare by value in filters; everything else compares by id for equality and by spelling for order.

5.2 A key whose bytes sort correctly

A B+ tree compares keys. The fastest comparison is `memcmp` on raw bytes, which works only if byte order equals the intended order. For a key made of three unsigned integers, that requires **big-endian** encoding: the most significant byte first.

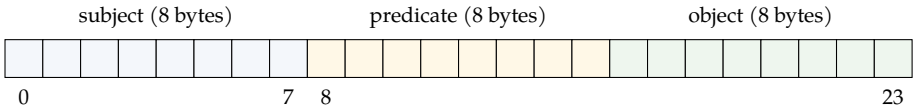


Figure 5.1: A 24-byte triple key: three big-endian unsigned 64-bit ids.

Why big-endian: little-endian stores the least significant byte first, so the id 256 (`00 01 00...` in little-endian) would compare below the id 1 (`01 00 00...`). With big-endian, integer order and byte order agree, and the comparison needs no decoding at all.

5.3 Prefix scans

A triple pattern with some positions bound is a *prefix* of the key, if the bound positions come first. "Every triple with subject 5" is every key whose first 8 bytes encode 5. Such a prefix is a contiguous range of keys, so a scan is one descent plus a walk along leaves. The range is $[\text{lower}, \text{upper})$, where lower is the prefix padded with zeros and upper is the prefix's last component plus one:

Bound components	Lower bound	Upper bound (exclusive)
0	`(0, 0, 0)`	past every key
1	`(5, 0, 0)`	`(6, 0, 0)`
2	`(5, 7, 0)`	`(5, 8, 0)`
3	`(5, 7, 9)`	`(5, 7, 10)`

5.4 One ordering is not enough

An index ordered subject, predicate, object (SPO) serves any pattern whose bound positions are a prefix of S-P-O. It does not serve "which subjects have object `Material`?": the object is the last component, so the matching keys are scattered across the whole tree. The fix is to store the same triples in more than one order.

Hexastore [13] keeps all six permutations; RDF-3X [14] also keeps six but compresses them heavily and adds aggregated indexes. Each permutation is another full copy of the triples, so the right number is a measurement, not a default: three cover every pattern as a prefix, and the other three only add sorted orders that some joins can exploit.

Bound positions in the pattern	Answered as a prefix of
subject	SPO
subject, predicate	SPO
predicate	POS
predicate, object	POS
object	OSP
object, subject	OSP
all three	any (a point lookup)
none	any (a full scan)

Table 5.1: Three permutations (SPO, POS, OSP) make every bound/unbound combination a prefix scan.

Key idea

Extra permutations change the *cost* of answering a pattern, never the *answer*. With only SPO, every pattern is still answerable by scanning and filtering; the other permutations turn scans into prefix ranges.

In OntoDB: keys, permutations, and the catalog

OntoDB's `TripleKey` is exactly fig. 5.1: three big-endian `uint64`s, compared with `memcmp`. `PrefixLower(key, n)` and `PrefixUpper(key, n)` produce the bounds for $n = 0, 1, 2, 3$ bound components. The upper bound adds one to the prefix, read as a big-endian number, carrying left. When nothing lies above the prefix ($n = 0$, or every bound id is the maximum) it saturates to the all-`0xFF` key, three copies of `INVALID_TERM_ID`, which the dictionary never assigns. The catalog names three B+ tree roots, `"spo"`, `"pos"`, and `"osp"`, and an insert or delete updates every permutation the catalog has, or none. The plan builds SPO first and then *measures* whether POS and OSP pay for their space on real SPARQL workloads before adding them; the next design decision records that measurement.

Design decision 30: Ids start at 1; 0 and the maximum are reserved

Choice. Term ids are dense 64-bit integers starting at 1. Id 0 is never used. The largest id, `INVALID_TERM_ID` ($2^{64} - 1$), is the engine-wide marker for "unbound": a row slot that no pattern has filled, a pattern position that is a variable. `Dictionary::CheckAssignable` refuses to hand it out, so every dictionary enforces the reservation, not just the arithmetic.

Alternatives. An `optional` per row slot (a flag beside every id), or a separate bitmap of which slots are bound.

Why. A row stays a plain array of `uint64`, which joins compare and hash directly. And because no stored triple can contain the maximum id, the all-`0xFF` key lies above every real key, which the prefix bounds use.

Cost. Two ids out of 2^{64}, and a contract every dictionary must keep; a dictionary that skipped the check could hand a real term an id that every operator reads as "unbound".

Design decision 31: The dictionary is built from the engine's own pages

Choice. On disk, spellings live in the term heap. Two B+ trees index them, both with ordinary 24-byte triple keys: one keyed $(id, page, slot)$, so a prefix scan on the id finds the record; one keyed $(hash, id, 0)$, so a prefix scan on a spelling's hash lists candidate ids, each checked against the heap. A header page holds the term count and the three page ids.

Alternatives. Keep the dictionary in memory and rebuild its hash maps from the heap at every open; or write a dedicated on-disk hash index, such as extendible hashing.

Why. It reuses parts that are already tested, works when the dictionary is larger than memory, and needs nothing rebuilt at startup. Comparing the spelling makes hash collisions harmless.

Cost. Looking up a spelling is two tree descents and a heap read, against one probe of an in-memory hash table, and each term costs 48 bytes of index on top of its spelling. A memory cache in front is the obvious optimization if lookups dominate a load.

Design decision 32: A fixed hash function

Choice. The spelling hash is 64-bit FNV-1a, written out in the code.

Alternatives. `std::hash`, or a faster modern hash from a library.

Why. A hash stored on disk must give the same value in every build, on every machine, forever. `std::hash` promises none of that; a different standard library would silently make every stored entry unreachable.

Cost. FNV-1a is slower and mixes less well than newer hashes; with spellings this short, and collisions resolved by comparison, neither matters.

Design decision 33: Big-endian keys, compared with `memcmp`

Choice. Each id is written most significant byte first, so comparing the 24 bytes with `memcmp` gives numeric (s, p, o) order. `Compare` returns the sign of `memcmp`.

Alternatives. Store native integers and compare by decoding three `uint64`s; or store little-endian and use a custom comparator.

Why. Pages, binary search, and the iterator can treat keys as opaque bytes, with no knowledge of their structure. Note the contrast: the pageLSN and log records use the machine's own byte order (little-endian on x86 and ARM), because nothing ever compares them as bytes.

Cost. A byte swap on every encode and decode on little-endian machines, which is negligible.

Design decision 34: The upper bound saturates instead of failing

Choice. `PrefixUpper` adds one to the prefix with carry. When there is nothing above the prefix ($n = 0$, or every bound id is the maximum), it returns the all-`0xFF` key rather than an error or a flag.

Alternatives. Return an optional bound meaning "to the end of the tree", or a flag the

iterator checks.

Why. One key type for both ends keeps the scan loop to a single test: stop when the key reaches upper. Saturation is safe because of the reserved maximum id above.

Cost. The scan's correctness now depends on the dictionary's contract, which is why that contract is checked in code and tested.

Design decision 35: The catalog points at header pages, not roots

Choice. The catalog, page 0, maps a name (`"spo"`) to the page id of that tree's *header* page; the header page holds the tree's current root.

Alternatives. Store each tree's root page id in the catalog directly.

Why. A root changes whenever the root splits or collapses. With a header page in between, only the tree's own header page changes, and the catalog is written only when an index is created or dropped.

Cost. One more page read to find a root, though the header pages are small in number and stay in the pool.

Design decision 36: The catalog claims page 0 by writing it

Choice. On first open, the catalog finds page 0 all zeros, writes a magic number, an empty table, and an `INVALID_PAGE_ID` dictionary root, and writes the page to the file immediately. A page 0 with other contents is refused, not reinterpreted.

Alternatives. Allocate the catalog like any other page and remember its id somewhere, or trust that page 0 is the catalog.

Why. The disk manager's allocation record is the file's length. Writing page 0 at once makes the file one page long, so no later allocation can hand out id 0 to something else. The magic number distinguishes "never initialized" from "a catalog whose dictionary root happens to be 0".

Cost. An extra page write when a database is created.

Design decision 37: SPO, POS, and OSP, measured

Choice. The default store keeps three permutations: SPO, POS, and OSP.

Alternatives. SPO alone, which answers everything with a scan and a filter; or all six, as Hexastore keeps.

Why. Measured on LUBM(1), about 100,000 triples, with each pattern shape run on a cold pool of 64 frames (table 5.2). With SPO alone, every pattern that binds the predicate or the object but not the subject reads the whole tree, and `?x rdf:type C`, LUBM's most common pattern, is one of them. POS and OSP turn those into range scans, 9 to 45 times fewer page reads. The other three permutations make no pattern cheaper.

Cost. Twice the space of SPO alone and about a quarter more load time. SOP, PSO, and OPS would be worth their space only to an operator that needs a second sort order, such as a merge join.

Permutations	Load (s)	File (MiB)	`-p-`	`-o`	`-po`
SPO	1.4	8.2	1131	1131	1131
SPO, POS, OSP	1.7	16.2	128	25	25
all six	2.9	29.1	143	25	25

Table 5.2: Page reads per pattern on a cold 64-frame pool, LUBM(1), averaged over 20 sampled patterns. The other five shapes cost the same under all three sets. Full results are in `benchmarks/indexing/results.md`.

Design decision 38: Permutations are data, and every write is all-or-nothing

Choice. An index is named by its key order (`"spo"`, `"pos"`, ...), and the store keeps whichever of the six the catalog lists; a fresh catalog gets a default set. An insert or delete goes to every index in turn, and if one throws, the indexes already changed are put back before the error propagates. A scan uses the index with the longest bound prefix and filters the remaining bound positions.

Alternatives. Hard-code SPO, POS, and OSP as three fields of the store; or write to the indexes without undoing a partial write.

Why. The number of permutations is exactly the decision the next section measures, so it must be a knob, not a rewrite. A triple present in one index and missing from another would make answers depend on which index a query happened to use.

Cost. Every write costs one tree insert per index. The undo is in-memory only: it does not survive a crash, which is the log's job (chapter 11).

Design decision 39: The shell's indexed backend lives in a temporary file

Choice. `\set backend indexed` builds a buffer pool, the catalog, a disk dictionary, and the indexes in a temporary database file, and moves every triple across by its spelling. `\set backend mem` moves them back and deletes the file. While the indexed backend is on, statements run one at a time.

Alternatives. Make the page store the database behind `ontodb <dir>`.

Why. A persistent page store needs page-level recovery (plans 8.3 to 8.7) to survive a crash; until then, persistent databases use the memory store and its logical log. A temporary file lets every query run on both backends today, which is what the differential test does, and the tree is single-threaded until latch crabbing (plan 7.1).

Cost. Indexed data disappears when the shell exits, ids differ between backends, and two sessions cannot run statements at the same time.

Status

The triple key runs (plan 0.2): encode, decode, compare, and both prefix bounds, with the carry and saturation cases tested. So do the catalog (plan 3.3), the disk dictionary (plan 3.2), and the indexed store (plan 3.4): every query file in the test suite returns the same rows on the memory store and on the B+ tree store, and `\set backend indexed` switches the shell to it. The permutation set was measured (plan 3.5), and a store flushed and reopened

keeps its triples and terms (plan 3.6).

Check your understanding

5.1 Encode the id 258 as 8 big-endian bytes and as 8 little-endian bytes. Which encoding sorts correctly under `memcmp` against the id 3?

5.2 Give the lower and upper bounds for the pattern `(5, ?, ?)` and for `(5, 7, ?)`.

5.3 What is `PrefixUpper` of the key $(5, 2^{64} - 1, 9)$ with $n = 2$? With $n = 1$ and subject $2^{64} - 1$?

5.4 Which permutation serves `?s ?p <Material>`? Which serves `<Alice> ?p <Student>`?

5.5 Why must the loader and the binder share one canonical spelling for terms? Give a literal that two careless implementations might spell differently.

5.6 What would go wrong if the dictionary assigned $2^{64} - 1$ to a real term?

5.7 In table 5.2, why does adding SOP, PSO, and OPS not reduce the page reads of any pattern? What kind of operator would make them worth their space?

5.8 With only the SPO index, which of the eight bound/unbound patterns are answered by a range scan, and which by a full scan plus a filter?

5.9 Two different spellings have the same 64-bit hash. What does the disk dictionary do when you look up the second one?

5.10 Why does the catalog record each index's header page rather than its root? How often does the catalog page change?

Answers are in appendix B.

Further reading

Weiss, Karras, and Bernstein [13]; Neumann and Weikum [14], the sections on the dictionary and on permutations.

Chapter 6

The Log-Structured Alternative

A B+ tree updates pages *in place*: an insert finds the right leaf and changes it. The other great family of storage engines never updates in place. A *log-structured merge tree* (LSM tree), introduced by O'Neil et al. [11], turns every write into a sequential append and pays for it later, in the background. RocksDB, LevelDB, Cassandra, and many modern stores are LSM trees. Knowing why OntoDB does *not* use one teaches as much as knowing why it uses a B+ tree.

6.1 How an LSM tree works

1. **Memtable.** Writes go to an in-memory sorted structure (often a skip list), and to a write-ahead log for durability.
2. **Flush.** When the memtable is full, it is written to disk as an immutable sorted file, an *SSTable*, in one sequential write.
3. **Compaction.** SSTables accumulate. A background process merges them into larger SSTables, dropping overwritten and deleted entries. In *leveled* compaction, level $L + 1$ is about ten times larger than level L.
4. **Reads** check the memtable, then SSTables from newest to oldest, and stop at the first hit.
5. **Deletes** write a *tombstone*, a marker that hides older versions until compaction removes both.

A read that misses must look in many files. *Bloom filters*, one per SSTable, answer "is this key possibly here?" in memory with no false negatives, so most files are skipped without a read.

6.2 Three amplifications

Every storage engine trades three costs against each other.

Write amplification Bytes written to the device per byte of user data. An LSM tree rewrites data at every compaction; a B+ tree rewrites a whole page to change a few bytes.

Read amplification Pages read per lookup. A B+ tree reads one path; an LSM tree may consult several levels.

Space amplification Bytes stored per byte of live data. LSM trees hold obsolete versions until compaction; B+ trees hold half-empty pages.

	B+ tree	LSM tree
Writes	random, in place	sequential, batched
Point reads	one root-to-leaf path	memtable, then levels (Bloom filters help)
Range scans	walk linked leaves	merge iterators across levels
Deletes	remove now, merge pages	tombstone now, remove at compaction
Background work	none required	compaction, always
Best for	read-heavy, range-heavy	write-heavy, ingest-heavy

Table 6.1: The two families, in broad strokes.

Key idea

An LSM tree buys fast writes with deferred work. A B+ tree pays for each write immediately, and gets predictable reads and no background compaction in return.

In OntoDB: why a B+ tree

OntoDB's workload is a SPARQL engine's: a bulk load, then queries that are joins of prefix range scans. Range scans over sorted permutations are a B+ tree's best case, and bulk loading builds full, sequential leaves without any compaction. A delete in OntoDB's tree shrinks the tree in place, inside `BPlusTree::Remove`; there is no tombstone for a reader to skip and no background job that rewrites sorted runs.
The LSM ideas still appear in OntoDB, in smaller roles. Its persistent databases (chapter 13) are log-plus-snapshot: every change is appended to a log, and a checkpoint rewrites the whole state as one sorted image, which is a compaction with a single level. And Bloom filters are a natural fit for "has this concept been modified recently?" checks over a log.

Check your understanding

6.1 A workload inserts 10 million triples per hour and runs few queries. Which family fits better, and what does the other one pay?

6.2 Why does an LSM delete write a tombstone instead of removing the key?

6.3 A Bloom filter says "maybe present" for a key that is absent. What does that cost an LSM read? Can it say "absent" for a present key?

6.4 In what sense is OntoDB's snapshot-plus-log a one-level LSM tree?

Answers are in appendix B.

Further reading

O'Neil et al. [11]; Petrov [1], chapter 7; Bloom [12].

Part III

Query Processing

Chapter 7

From SPARQL to a Plan

A query states *what* to find. The engine must decide *how*. This chapter follows a SPARQL query from text to an executable tree of operators, and introduces the iterator model that runs it.

7.1 Parsing and binding

The *parser* turns text into an abstract syntax tree: prefixes, the projected variables, the triple patterns, the filters. Syntax that the engine does not support should fail here, with a line and a column, rather than half work later.

The *binder* then resolves names against the database:

- Prefixed names become full IRIs: `ub:Student` becomes `<http://example.edu/univ#Student>`.
- Every constant becomes a dictionary id. For a query, a constant that is not in the dictionary can match nothing; for an insert, it is added.
- Every variable gets a *slot*: a column number, assigned in order of first appearance.

After binding, a row is simply an array of ids indexed by slot. The same variable in two patterns is the same slot, and that shared slot is the join key.

7.2 Logical and physical plans

A *logical plan* says what to compute in terms of algebra: scan each pattern, join the results, filter, project. A *physical plan* picks an algorithm for each step: which index and range to scan, which join algorithm, in what order. The planner produces a correct physical plan; the optimizer (chapter 8) rewrites it into a fast one that returns the same rows.

For the query

```
SELECT ?name WHERE {
  ?s a ub:Student . ?s ub:name ?name . ?s ub:age ?age .
  FILTER (?age > 21)
}
```

a straightforward plan is left-deep, joining the patterns in the order written:

```
Projection [?name]
  Filter (?age > 21)
    NestedLoopJoin
      NestedLoopJoin
```

```
      TripleScan {?s rdf:type ub:Student}
      TripleScan {?s ub:name ?name}
    TripleScan {?s ub:age ?age}
```

7.3 The iterator (Volcano) model

Graefe's *Volcano* model [15] gives every operator the same three-call interface:

- `Init()`: prepare, or restart from the beginning.
- `Next(&row)`: produce the next row and return true, or return false when exhausted.
- destruction: release everything held (for a scan, its pinned leaf).

The root operator calls `Next` on its child, which calls `Next` on its children, and so on down to the scans. Rows flow up *one at a time*, on demand. Three properties make this model the default for decades:

Composability. Any operator can sit above any other, because they share an interface.

Pipelining. A row can travel from a scan to the client without any intermediate result being materialized. `LIMIT 10` stops the whole tree after ten rows.

Bounded memory. Only operators that must see all their input first (sort, hash-join build, aggregation) hold rows.

Its cost is one virtual call per row per operator. Modern analytical engines reduce that with *vectorized* execution (batches of rows per call) or by compiling the plan into tight loops, but the logical structure is unchanged.

Key idea

Planning builds a tree; execution pulls rows through it. Keeping the two separate is what allows `\explain` to print a plan without running it, and an optimizer to swap one tree for another.

7.4 The operators

Triple scan. Reads the triples that match one pattern. With an index, the bound positions choose a permutation and a prefix range (chapter 5). If a variable occurs twice in one pattern (`?x ex:knows ?x`), the scan also checks that those positions are equal.

Join. Combines rows from two children whose shared slots agree (chapter 8).

Filter. Passes rows whose expression is true. In SPARQL a type error makes the expression false, not the query fail.

Projection. Keeps the selected slots.

Distinct, limit, offset, sort. Distinct applies to *projected* rows (two triples can share a name). Sort compares values, not ids: ids are assigned in load order, which is not value order.

7.5 Sorting more than fits in memory

A sort must see all its input before it can emit the first row, so it is the one common operator whose memory grows with its input. When the rows do not fit in the memory the sort may use, an *external merge sort* spills to disk in two phases.

1. **Run generation.** Read as many rows as fit in B pages of memory, sort them, and write them out as a sorted *run*. Repeat until the input is exhausted: N pages of input become $\lceil N/B \rceil$ runs.
2. **Merging.** Read the first page of up to F runs at once, and repeatedly emit the smallest row at the front of any of them. One merge turns F runs into one. Repeat until a single run remains, or until few enough remain that the last merge can stream its rows straight to the operator above.

With fan-in F, the number of merge passes is about $\lceil \log_F \lceil N/B \rceil \rceil$, and each pass reads and writes every page once. Doubling memory saves at most one pass; the real lever is that F is large, so most sorts need a single merge.

In OntoDB: planning and execution

OntoDB's binder assigns slots in first-appearance order and represents "not yet bound" with a reserved id. The planner always produces the left-deep tree in textual order with nested-loop joins, which is correct but rarely fast; `\explain` prints it. An executor factory maps each plan node to its executor, and the engine pulls rows with `Init/Next`. An empty pattern, with no triple to match, returns one empty row rather than none: the identity input for a join that has nothing to bind yet.

Design decision 40: Unsupported syntax fails at parse time

Choice. `OPTIONAL`, `UNION`, property paths, aggregates, and `DELETE/INSERT WHERE` are recognized by the parser and rejected there with a line and column.

Alternatives. Accept the full grammar and fail in the executor; or skip what is not understood.

Why. A query either runs completely or fails before anything happens. Ignoring an `OPTIONAL` would silently change the answer, which is worse than refusing.

Cost. The parser must know the shape of features the engine does not implement.

Design decision 41: Volcano, one row at a time

Choice. Every operator implements `Init` and `Next` and passes one row of ids per call.

Alternatives. Vectorized execution (a batch of rows per call), or compiling the plan into loops.

Why. It is the simplest model that composes, it makes `LIMIT` stop the whole tree, and it keeps pin ownership obvious: a scan holds one leaf while it has a current row.

Cost. A virtual call per row per operator. Large analytical scans would be several times faster vectorized.

Design decision 42: Stub plan nodes before their executors

Choice. `Sort`, `Distinct`, `Limit`, `Insert`, and `Delete` exist as plan nodes that `\explain` prints before their executors are written.

Alternatives. Add a plan node only together with its executor.

Why. The plan's shape and the planner's tests can be fixed first; each executor then arrives into a slot that is already wired and printed.

Cost. For a while, `\explain` showed plans that could not run, and running one threw "not built yet". Since plan 4.6 every node has its executor; insert and delete stopped being a special case in the engine and became an operator like the rest.

Design decision 43: `ORDER BY` uses a total order on values

Choice. Sort keys compare by value: unbound before blank nodes, before IRIs, before literals; numeric literals first among literals, by value; other literals by lexical form, then datatype, then language; and any remaining tie by id. Rows with equal keys keep their input order.

Alternatives. Compare ids, which is fast and meaningless; or use the filter comparisons, which call some pairs incomparable.

Why. Ids are assigned in load order, so sorting by id sorts by when a term was first seen. A sort needs every pair ordered, which the filter's SPARQL semantics do not give (comparing an IRI with a number is a type error there). Parsed terms are cached for the length of one sort.

Cost. A dictionary lookup and a parse per distinct value sorted. Without a dictionary (some unit tests), the sort falls back to ids.

Design decision 44: External sort through the buffer pool

Choice. A sort may use half the buffer pool, at least four pages. If its input fits, it sorts in memory. Otherwise it writes sorted runs to chains of pool pages, merges up to (budget − 2) runs at a time, and streams the last merge to its consumer, choosing the earlier run on ties so the sort stays stable. Each run page is freed as soon as it has been read. On the memory backend there is no pool, and every sort is in memory.

Alternatives. Sort in memory always; or spill to temporary files outside the database, as many engines do.

Why. The pool already bounds memory and does the I/O, so the sort's memory use is a fraction of a number the user set (`pool_size`), and its pages are ordinary pages that tests can count.

Cost. Spilled pages take new page ids, and the file only grows (chapter 2): every external

sort leaves the database file larger, permanently. A sort also competes with the query's own pages for frames.

Design decision 45: `DISTINCT` keeps a hash set

Choice. The distinct operator passes a row the first time it sees it, in its child's order, and remembers every row it has passed in a hash set.

Alternatives. Sort the rows and drop adjacent duplicates.

Why. Hashing streams: the first answer comes out as soon as it is found, `LIMIT` above it still stops early, and the order the child chose is kept.

Cost. Memory grows with the number of distinct rows. A sort-based distinct could spill to disk; this one cannot.

Design decision 46: Rows have no order without `ORDER BY`

Choice. The engine promises no order unless the query has `ORDER BY`. Tests compare answers as multisets.

Alternatives. Always return rows in a fixed order, for example sorted by id.

Why. The order a plan produces depends on the store, the indexes, and the join operator: the memory store returns triples in insertion order, the indexed store in key order. Promising an order would force a sort on every query.

Cost. `LIMIT` without `ORDER BY` may return different rows on the two backends, or under different join operators. Both answers are correct, which is why the differential test checks such queries only when the limit is larger than the answer.

Status

Parsing, binding, planning, triple scans, the three joins, filter, projection, distinct, limit, offset, sort, insert, and delete run on both backends as operators (plans 4.1 to 4.6). On the indexed backend a sort larger than half the pool spills to pages; a test sorts 3,000 rows on an 8-frame pool through three runs and one intermediate merge.

Check your understanding

7.1 In `?s a ub:Student . ?s ub:name ?name . ?s ub:age ?age`, which slot does each variable get?

7.2 Why does `DISTINCT` apply to projected rows, not to the rows the scans produce?

7.3 Why can't `ORDER BY` compare term ids?

7.4 A plan is `Limit 10` over a join of two large scans. How much work does the Volcano model do, and how much would a model that materializes each operator's output do?

7.5 Why does an empty pattern produce one empty row rather than none?

7.6 A sort's input is 1,000 pages and its budget is 10 pages, so its fan-in is 8. How many runs

does run generation write, how many merge passes follow, and how many pages are written in all?

7.7 `SELECT ?name WHERE { ... } LIMIT 2` returns Alice and Bob on the memory backend and Alice and Cara on the indexed one. Which is wrong?

Answers are in appendix B.

Further reading

Graefe [15]; the W3C SPARQL 1.1 recommendation [18].

Chapter 8

Joins and Query Optimization

A SPARQL query of k triple patterns is a k-way join, and its running time depends far more on *how* it is joined than on anything else. This chapter covers the three join algorithms, how an optimizer estimates the size of intermediate results, and how it searches for a good join order.

8.1 Three join algorithms

Let the outer input have $|R|$ rows and the inner input $|S|$ rows.

Nested-loop join. For each outer row, scan the whole inner input and keep the pairs whose shared slots agree. Cost: $|R| \times |S|$ comparisons, no extra memory. Correct for any join, and the fallback when nothing better applies.

Index nested-loop join. For each outer row, substitute its bindings into the inner pattern and *probe an index* for just the matching triples. If the outer row binds the subject and SPO exists, the inner side becomes a prefix range, often one or a handful of triples. Cost: $|R|$ index probes. This is the common, fast case in SPARQL: one pattern finds the entities, and each further pattern looks up an attribute of each entity.

Hash join. *Build* a hash table on the shared slots of one input (ideally the smaller), then *probe* it with each row of the other. Cost: linear in $|R| + |S|$, plus memory for the build side. It wins when both inputs are large and no index makes the probe selective, and it loses when the build side does not fit in memory, or when an index probe was already a point lookup.

Key idea

Index nested loop is fast when the outer side is small and the inner side is indexed on the join key; hash join is fast when both sides are large; nested loop is always correct. An optimizer's job is to know which case it is in.

8.2 Cardinality estimation

To compare plans, the optimizer needs the *cardinality* of each intermediate result: how many rows each operator produces. It estimates them from *statistics* gathered at load time: the total triple count $|T|$, the number of distinct subjects, predicates, and objects, and per-predicate counts.

For a pattern with predicate p and object o bound, the classic estimate assumes the positions are *independent*:

$$|\{\widehat{(s,p,o)}\}| \;=\; \frac{|p|\cdot|o|}{|T|},$$

where $|p|$ and $|o|$ are the counts of triples with that predicate and that object. Independence is convenient and frequently wrong. In RDF, predicates and objects are strongly correlated (`rdf:type` goes with class objects; a `name` predicate never has a class as its object), so independence can be off by orders of magnitude.

The *q-error* measures an estimate $\hat{c}$ against the true count c:

$$q(\hat{c},c) = \max\left(\frac{\hat{c}}{c},\ \frac{c}{\hat{c}}\right) \geq 1.$$

It treats over- and underestimates symmetrically, and errors multiply as they propagate up a plan: a join of two inputs each off by 10 can be off by 100.

8.3 Searching for a join order

A left-deep plan over k patterns has $k!$ orders, and each join can use several algorithms. Selinger's System R optimizer [16] made the search tractable with dynamic programming:

```
for each single pattern P:   best[{P}] = cheapest access path for P
for size = 2 to k:
  for each subset S of patterns with |S| = size:
    for each pattern P in S:
      candidate = join(best[S - {P}], P) with the cheapest algorithm
      if cost(candidate) < cost(best[S]): best[S] = candidate
return best[all patterns]
```

It keeps one best plan per subset rather than per order, which reduces $k!$ to about $k \cdot 2^k$. A simpler *greedy* strategy (start with the most selective pattern, then repeatedly add the cheapest connected pattern) is a good first step and often good enough for the star-shaped joins common in SPARQL.

8.4 Explain analyze

The single most useful optimizer tool is to *run* the plan and print, next to each operator's estimated rows, the rows it actually produced. The node where the two diverge most is where a statistic is missing, usually a correlation that the independence assumption averaged away.

In OntoDB: the optimizer

OntoDB's optimizer takes the planner's tree and returns another tree that must return the same rows. The shell setting `\set join auto|nlj|inlj|hash` either lets it choose or forces one algorithm, and forcing happens in the optimizer, not the executor, so the plan that `\explain` prints is always the plan that runs. Statistics are collected at load: the triple count,

distinct counts per position, per-predicate counts, and counts for partially bound patterns. Estimates are finite, never negative, and binding one more position never increases them. Join ordering is greedy and left-deep.

Design decision 47: Exact counts for single patterns

Choice. Statistics are collected in one scan of the store: counts per subject, predicate, and object, per pair of positions, and the set of triples. Every single triple pattern of constants therefore has an *exact* cardinality; estimation, and its independence assumptions, begin only at joins.

Alternatives. Distinct counts and per-predicate counts only, combined with the independence formula above; histograms or samples; or RDF-3X's characteristic sets, which summarize which predicates occur together on one subject.

Why. The independence formula is wrong by orders of magnitude on RDF, where predicates and objects are correlated. For a store of this size, exact pair counts are affordable and remove that error from every scan estimate.

Cost. About seven hash entries per triple, collected at load, and stale after updates until the next collection, like a table that has not been re-`ANALYZE`d.

Design decision 48: Join estimates use distinct values

Choice. A join's estimate is System R's $|L|\,|R| / \max(V(L,x), V(R,x))$ for each shared variable x, where V is the number of distinct values x takes. For a single pattern, V comes from statistics: a predicate's distinct subjects or objects, or the pattern's own count when the other positions are bound.

Alternatives. Assume each side's join values are all distinct, which gives $\min(|L|, |R|)$ for one shared variable; or a fixed selectivity per join.

Why. SPARQL joins come in two shapes that the simple rule cannot both get right. On a store of 100 students taking 3 of 10 courses, joining students to their names (a star on the subject) is 100 rows, and joining enrolments to the course class is 300. Distinct values give both exactly; $\min(|L|, |R|)$ says 10 for the second, 30 times too low.

Cost. Two more counts per predicate, and an assumption that remains: the join values of the smaller side are taken to appear on the larger side. A variable's V in an intermediate result is approximated by the smaller of its V in a pattern and the result's size.

Design decision 49: The inner side is re-opened with the outer row

Choice. Every executor has `InitWith(row)`, which by default is `Init`. A triple scan overrides it: each variable that the outer row already binds becomes a constant in the pattern it hands the store. The nested loop re-opens its inner side this way for every outer row.

Alternatives. Open the inner scan once with its own pattern and compare every inner row with every outer row; or materialize the inner side once and loop over the copy.

Why. With the outer row filled in, the inner pattern usually binds the subject, so on an indexed store each probe is a short prefix range instead of a scan of every triple with that

predicate. The cost per outer row drops from the size of the inner pattern to the size of its match.

Cost. One store scan opened, and one tree descent, per outer row, with no caching between rows that bind the same values.

Design decision 50: Index nested loop probes the store directly

Choice. The index nested-loop join requires its inner side to be one triple pattern. It has no inner executor: for each outer row it builds the probe pattern from the outer row and asks the store, which picks the index whose order starts with the bound positions.

Alternatives. Treat any nested loop over a rebound scan as an index nested loop, since with rebinding it already probes.

Why. With rebinding, the nested loop already probes an index, so the two operators read the same triples. The separate operator skips the inner executor and the generic row merge, and it names the strategy in the plan, where `\explain` and the optimizer can see it.

Cost. Only a scan can be the inner side; a join whose inner side is itself a join stays a nested loop.

Design decision 51: Hash join builds on the inner side

Choice. The hash join reads its inner (right) input once into a table keyed by the ids of the variables both sides produce, then streams the outer input through it. A row whose key slot is unbound is not hashed but compared with every row of the other side.

Alternatives. Build on whichever side is smaller, or partition both sides to disk when the table does not fit (a grace hash join).

Why. Choosing the smaller side needs cardinality estimates, which arrive with the optimizer (plans 5.1 and 5.2); until then the inner side of a left-deep plan is a single pattern, usually the smaller input.

Cost. The whole inner side lives in memory; a large inner pattern can make the build expensive or exhaust memory. There is no spilling.

Design decision 52: Forcing a join method rewrites the plan

Choice. `\set join hash` (or `nlj`, `inlj`) is applied by the optimizer (`Optimizer::ApplyJoinMethod`), which swaps each nested-loop node for the chosen one before the plan is explained or run. The executor factory only reads node types.

Alternatives. Keep one join node and let the executor read the setting at run time.

Why. `\explain` then always prints the plan that runs. A test can check that a forced hash plan contains a hash join without running it.

Cost. The optimizer must be invoked even when the user forces everything.

Design decision 53: Greedy, left-deep join ordering first

Choice. Start from the pattern with the fewest estimated rows and repeatedly add the pattern connected to what is already joined (sharing a variable with it) whose join is estimated smallest, producing a left-deep tree. A cross product is taken only when no remaining pattern is connected, and ties keep the order the query was written in. The optimizer runs whenever statistics exist, which in the shell is always.

Alternatives. Selinger's dynamic programming over subsets, or a search that also considers bushy trees.

Why. Most SPARQL basic graph patterns are stars around one subject, for which greedy is close to optimal, and a greedy order is easy to explain in `\explain analyze`.

Cost. Chains and snowflakes, where a bushy plan or a non-greedy first step wins, can be ordered badly.

Design decision 54: `EXPLAIN ANALYZE` wraps every executor

Choice. `\explain analyze` builds the plan's executors with each one wrapped in a counter, runs the query to the end, and prints every node with the optimizer's estimate beside the rows it actually returned. The inner scan of an index nested-loop join is marked "probed by the join above", since the join asks the store directly and the scan never runs.

Alternatives. Count only at the root; or sample timings per operator.

Why. The node where estimate and actual diverge most is where a statistic or an assumption failed, and only per-node counts show which node that is. Wrapping keeps the executors unaware of the measurement.

Cost. One extra virtual call per row per operator, and the query really runs: `explain analyze` of a slow query is slow. Updates are planned but not run.

Design decision 55: `auto` picks the operator per join, and is the default

Choice. With `\set join auto`, the default, each join of the reordered chain chooses its own operator: a cross product stays a nested loop; otherwise an index nested-loop join when the estimated outer side is no larger than the inner pattern, and a hash join, built on the smaller inner pattern, when it is larger.

Alternatives. Cost every operator with a formula for each and take the cheapest; or keep nested loop as the default and let users force the rest.

Why. With rebinding and indexes, a probe per outer row is cheap while the outer side is small, and a hash table pays off once the outer side outgrows the pattern it would probe. On LUBM (table 8.1) the optimizer's order matters far more than the operator, and `auto` is never worse than the best forced setting.

Cost. The rule compares row counts, not costs: it ignores that an index probe is a tree descent and a hash build is memory. It can be wrong near the crossover, where both choices cost about the same.

Query	Rows	written	nlj	inlj	hash	auto
Q1	4	2.4	0.0	0.0	0.3	0.0
Q2	0	$>$ 20 s	0.2	0.1	2.6	0.1
Q3	6	8.1	0.1	0.0	0.9	0.0
Q4	0	0.0	0.0	0.0	6.7	0.0
Q14	5916	8.3	7.6	7.8	7.9	7.7

Table 8.1: Milliseconds per LUBM query on LUBM(1), with the whole store in the pool. "written" is the query's own order without the optimizer; the other columns use the optimizer's order with the named operator. Many queries return nothing because they need inference. Query 2, written with three unconnected type patterns first, does not finish in 20 seconds without reordering. Full table: `benchmarks/joins/results.md`.

Status

Today the planner always joins in textual order with nested loop, and the nested loop rebinds its inner scan for every outer row (plan 4.1). The index nested-loop join (plan 4.2) and hash join (plan 4.3) run, chosen with `\set join`, and the differential test checks every query file under all three against the memory store. Statistics (plan 5.1), cardinality estimation (plan 5.2), and greedy join ordering (plan 5.3) run: a query written with `?s ?p ?o` first is reordered to start from its selective pattern, and `\explain` shows the reordered plan. The automatic choice of join (plan 5.4) runs and is the default, and `\explain analyze` prints estimated against actual rows for every node (plan 5.5): the optimizer is complete. A query can carry a deadline, which every scan checks; the joins benchmark uses it.

Check your understanding

8.1 Outer side 1,000 rows, inner pattern 1,000,000 triples, and the join binds the inner subject with SPO available. Compare nested loop and index nested loop.

8.2 When does a hash join beat index nested loop?

8.3 With exact pair counts, what is OntoDB's estimate for `(?, p, o)`, and where can its estimates still go wrong?

8.4 A store has $|T| = 10^6$, $|p| = 10^4$, $|o| = 10^3$. What does the independence formula estimate for `(?, p, o)`? The true count is 1,000. What is the q-error?

8.5 Why do estimation errors multiply up a plan?

8.6 A hash join builds on `?c a ub:Course` (2,000 rows) and probes with `?s ub:takesCourse ?c` (50,000 rows). What does it keep in memory, and how many hash lookups does it make? What changes if the sides are swapped?

8.7 `?s a ub:Student . ?s ub:name ?n`: the first pattern matches 1,000 students and the second 1,000,000 triples. How many triples does the nested loop read with rebinding on an indexed store, and how many without it?

8.8 Why does forcing a join method in the optimizer, and not in the executor, matter for `\explain`?

8.9 In table 8.1, query 2 takes over 20 seconds in its written order and 0.1 ms after reordering. What does its written order do first, and why can no join operator rescue it?

Answers are in appendix B.

Further reading

Selinger et al. [16]; Graefe [15]; Leis et al. [17] on how good cardinality estimates really are; Neumann and Weikum [14] on RDF selectivity.

Part IV

Transactions

Chapter 9

Transactions and Isolation

A *transaction* is a group of reads and writes that the database treats as one unit. Transactions let an application reason about one operation at a time, even though many run concurrently and any of them may be interrupted by a crash.

9.1 ACID

Atomicity. All of a transaction's writes happen, or none do. A transaction that aborts, or is cut off by a crash, leaves no trace. (Provided by the log: chapters 11 and 12.)

Consistency. A transaction takes the database from one valid state to another, where "valid" is defined by the application's invariants. The engine helps with constraints, but consistency is mostly the application's promise.

Isolation. Concurrent transactions do not see each other's intermediate states; the result is as if they ran in some order. (Provided by concurrency control: chapter 10.)

Durability. Once a commit is acknowledged, its effects survive any later crash. (Provided by the log and `fsync`.)

9.2 Schedules and serializability

A *schedule* is the interleaved order in which the operations of concurrent transactions actually execute. A schedule is *serializable* if its effect equals that of *some* serial order of the same transactions. Serializability is the gold standard of isolation: the application cannot tell that concurrency happened.

Two operations *conflict* if they come from different transactions, touch the same item, and at least one is a write. A schedule is *conflict-serializable* if it can be turned into a serial one by swapping adjacent non-conflicting operations. The test is a graph: draw an edge $T_i \to T_j$ for every pair of conflicting operations where T_i's comes first. The schedule is conflict-serializable exactly when the graph has no cycle.

9.3 Anomalies

Weaker isolation admits specific bad interleavings. Naming them is the clearest way to define the isolation levels [19].

Dirty read. T_2 reads a value T_1 wrote but has not committed. If T_1 aborts, T_2 read data that never existed.

Non-repeatable read. T_1 reads an item, T_2 changes it and commits, and T_1 reads it again and sees a different value.

Phantom. T_1 reads the set of items matching a condition ("every student"), T_2 inserts a new matching item and commits, and T_1 repeats the query and sees an extra row.

Lost update. Both transactions read a counter, both add one, both write: one increment disappears.

9.4 Isolation levels

Level	Dirty read	Non-repeatable read	Phantom
Read uncommitted	possible	possible	possible
Read committed	—	possible	possible
Repeatable read	—	—	possible
Serializable	—	—	—

Table 9.1: The SQL standard's levels, defined by the anomalies they forbid.

Berenson et al. [19] showed that defining levels by these phenomena alone is ambiguous, and that real systems' levels (notably snapshot isolation, which forbids all three but admits a "write skew") do not fit the table cleanly. The table remains the right first mental model.

Key idea

Isolation is a dial, not a switch. Lower levels allow more concurrency and specific anomalies; serializable allows none. Each level is defined by what it forbids, and each forbidden anomaly corresponds to a specific lock held for a specific time (chapter 10).

In OntoDB: transactions

Every shell statement is its own transaction unless `\begin` opens an explicit one, ended by `\commit` or `\abort`. Isolation is a setting, `\set isolation ru|rc|rr|ser`, defaulting to read committed. The isolation contract is written as executable schedules: twelve specs, each a scripted interleaving of two sessions with expectations such as "session 2 is blocked", "the read returns Dana", or "session 2 is aborted". They cover dirty reads, non-repeatable reads, lost updates, lock upgrades, two deadlock shapes, write–write conflicts, locks released on abort, and phantoms.
In a persistent database today, each update statement (or `\load`) is one atomic, durable transaction (chapter 13).

Design decision 56: Every statement is a transaction

Choice. Each update statement, and each `\load`, is one atomic and durable transaction unless `\begin` opens an explicit one.

Alternatives. Require an explicit `BEGIN`; or apply statements immediately with no atomic-

ity.

Why. An ontology edit is usually one statement, and a load should be all or nothing: a half-loaded OWL file is a corrupt ontology.

Cost. A very large `\load` is one very large transaction, logged and held in memory until its commit.

Design decision 57: A failed statement takes itself back

Choice. `INSERT DATA` and `DELETE DATA` run as one operator that remembers the triples it has changed. If a later write throws, it undoes those changes, newest first, before the error reaches the caller. The store, for its part, writes each triple to every index or none.

Alternatives. Leave a half-applied statement in place and rely on the caller; or rely on the log to undo it.

Why. A statement should be atomic on both backends, including the temporary indexed one, which has no log. Undo in memory is enough for an exception; the log is still needed for a crash.

Cost. The undo list lives only in memory: if the process dies in the middle of a statement, only the log can take it back (chapter 12), and the memory backend without a directory keeps nothing anyway.

Design decision 58: Read committed by default (planned)

Choice. `\set isolation` defaults to `rc`.

Alternatives. Serializable, or repeatable read (MySQL's InnoDB default).

Why. It matches PostgreSQL's default, forbids dirty reads, and releases shared locks at the end of each statement, so readers block writers only briefly.

Cost. Non-repeatable reads and phantoms are possible unless a session raises its level.

Design decision 59: Isolation is specified as executable schedules

Choice. The isolation contract is a set of spec files, each a scripted interleaving of two sessions with expected outcomes (blocked, returns a value, aborted).

Alternatives. Prose definitions, or randomized stress tests.

Why. Each anomaly becomes one small, deterministic scenario that either happens or does not. A stress test can pass by luck; a schedule cannot.

Cost. Only the interleavings someone wrote down are checked.

Status

Statement-level atomicity and durability run today: on every backend a failed statement undoes itself (plan 4.6), and in a persistent database every statement is a logged transaction. Explicit multi-statement transactions and isolation levels are plans 7.2 to 7.6.

Check your understanding

9.1 Draw the conflict graph of $r_1(A)\, w_2(A)\, r_2(B)\, w_1(B)$. Is the schedule conflict-serializable?

9.2 Give a schedule with a non-repeatable read, and say which isolation level first forbids it.

9.3 Why is consistency mostly the application's promise and not the engine's?

9.4 `INSERT DATA` names three triples, and the store throws on the third. What is in the store afterwards, on the memory backend without a directory?

9.5 Two sessions both run "read the count of `Students`, insert one more". Which anomaly can occur, and at which levels?

Answers are in appendix B.

Further reading

Gray and Reuter [3]; Berenson et al. [19]; Petrov [1], chapter 5.

Chapter 10

Concurrency Control

Isolation levels say *what* must hold; concurrency control is *how*. This chapter covers locking, the approach OntoDB takes: two-phase locking, multi-granularity locks, deadlock handling, phantoms, and the separate problem of protecting the B+ tree's own structure.

10.1 Two-phase locking

A transaction takes a *shared* (S) lock to read an item and an *exclusive* (X) lock to write it. S is compatible with S; X is compatible with nothing. *Two-phase locking* (2PL) adds one rule: a transaction may not acquire any lock after it has released one. Its life has a growing phase and a shrinking phase.

2PL guarantees conflict-serializability: the order in which transactions reach their *lock point* (the moment they hold all their locks) is an equivalent serial order.

Plain 2PL has a flaw. If T_1 releases an X lock before committing and T_2 reads the value, then T_1 aborts, T_2 has read a value that never existed, and must abort too: a *cascading abort*. *Strict 2PL* holds every X lock until commit or abort, which rules this out.

Key idea

Exclusive locks are held to the end of the transaction at every isolation level, because releasing one early lets others see a write that may still be undone. The isolation levels differ only in how long *shared* locks are held, and whether ranges are locked.

Level	Shared locks
Read uncommitted	none
Read committed	released at the end of each statement
Repeatable read	held until commit
Serializable	held until commit, plus range or predicate locks against phantoms

Table 10.1: Isolation levels as locking rules, given strict 2PL for writes.

10.2 Locking at several granularities

Locking every triple is precise but expensive for a query that reads the whole store; locking the whole store is cheap but serializes everything. Gray et al. [20] let a transaction choose its granularity, using *intention* locks on the coarser levels: before locking a key in S, take IS ("intends to share something below") on the index and on the database.

Held \ Requested	IS	IX	S	SIX	X
IS	yes	yes	yes	yes	—
IX	yes	yes	—	—	—
S	yes	—	yes	—	—
SIX	yes	—	—	—	—
X	—	—	—	—	—

Table 10.2: Multi-granularity lock compatibility. SIX is S on the whole node plus the intention to write below it.

The point of intention locks: a transaction that wants to lock a whole index in S can check one lock on the index, instead of every key below it, because any writer below must already hold IX there.

10.3 Deadlock

T_1 holds a lock T_2 wants, and T_2 holds a lock T_1 wants: both wait forever. Engines either *prevent* deadlock (order locks, or abort on conflicts by timestamp, as in wait–die and wound–wait) or *detect* it: maintain a *waits-for graph* with an edge $T_i \to T_j$ when T_i waits for a lock T_j holds, find a cycle, and abort one transaction in it, the *victim*. Choosing the youngest transaction (the one that has done the least work) is a common and deterministic policy.

10.4 Phantoms

Repeatable read locks the triples a transaction read. A phantom is a triple it did *not* read, because it did not exist yet; no lock covers it. Preventing phantoms needs a lock on what *could* match, not on what does:

Predicate locks lock the query's condition itself. Precise, but testing whether two predicates overlap is expensive in general [21].

Key-range (next-key) locks lock the gap between index keys, so an insert into the scanned range must wait. Practical, and natural on a B+ tree.

10.5 Latches are not locks

A B+ tree split rewrites several pages at once. Even a single-transaction engine with many threads must stop two threads from splitting the same page at the same time. That protection is a *latch*: a short-term reader–writer mutex on a page, held for the duration of one tree operation, never for a transaction. Locks protect *logical contents* for transactional isolation; latches protect *physical structure* for thread safety.

The classic latch protocol for B+ trees is *crabbing*: latch the child, and release the parent if the child is *safe*, meaning it cannot split (for an insert) or merge (for a delete) as a result of this

operation. A thread therefore holds latches only on the part of the path a structure change could reach. An *optimistic* variant takes shared latches down to the leaf, latches only the leaf exclusively, and restarts pessimistically if the leaf turns out to need a split. Lehman and Yao's B-link tree [7] goes further and lets readers run without latching the path at all, by following right-links when a split has moved their key.

10.6 The alternative: multiversion concurrency

Locking makes readers and writers wait for each other. *Multiversion concurrency control* (MVCC) keeps old versions of items, so a reader sees a consistent snapshot as of its start time and never waits for a writer. Writers still conflict with writers. The cost is storing old versions and cleaning them up once no reader can need them. Most modern systems use MVCC with snapshot isolation by default.

In OntoDB: concurrency control

OntoDB uses strict 2PL with five lock modes on three granularities: the database, an index (`"spo"`, `"pos"`, `"osp"`), and a key, with FIFO grant order and in-place upgrades that do not jump ahead of waiters. A blocked shell session prints why it is waiting, for example `session 2 waiting: X lock held by txn 5`. A background thread builds the waits-for graph and aborts the *youngest* transaction in each cycle (the larger id), so deadlock outcomes are deterministic and testable. Serializable adds one phantom mechanism, chosen and documented in the code. B+ tree operations use latch crabbing, and the whole feature runs under ThreadSanitizer. MVCC is a deliberate future comparison, not part of the plan.

Design decision 60: Locking, not multiversioning (planned)

Choice. Strict two-phase locking with multi-granularity locks.

Alternatives. MVCC with snapshot isolation, as PostgreSQL and most modern engines use; or optimistic concurrency control.

Why. Under locking, each anomaly is prevented by one specific lock held for one specific time, which makes isolation levels concrete and testable. It also pairs directly with ARIES, which was designed for it.

Cost. Readers and writers wait for each other. MVCC is planned as a measured comparison: what a reader no longer waits for, and what keeping old versions costs.

Design decision 61: Detect deadlocks and abort the youngest (planned)

Choice. A background thread builds the waits-for graph and aborts the transaction with the largest id in each cycle.

Alternatives. Prevention by wait–die or wound–wait, or lock-wait timeouts.

Why. Detection aborts only when a cycle really exists. Choosing the youngest is deterministic, so a deadlock spec can name the victim, and it usually discards the least work.

Cost. A detector thread, and transactions sit blocked until it runs.

Design decision 62: The phantom mechanism is a written choice (planned)

Choice. Serializable must block an insert into a range another transaction scanned, using key-range, gap, or predicate locks; whichever is chosen is written above the code, and repeatable read does not take it.

Alternatives. Lock the whole index for every scan at serializable.

Why. The difference between repeatable read and serializable should be exactly one mechanism, visible in one place.

Cost. Range locks interact with splits and merges, which is why this is the last slice of the feature.

Status

Concurrency control is plans 7.1 to 7.6: crabbing, transactions, the lock manager, isolation, deadlock detection, and phantoms. The isolation harness already exists and checks itself.

Check your understanding

10.1 Why does plain 2PL allow cascading aborts, and how does strict 2PL prevent them?

10.2 A transaction wants to read every triple in the `"spo"` index. Which locks does it take, and why is checking one lock enough to know no writer is below?

10.3 T_3 waits for T_5, and T_5 waits for T_3. Which one does OntoDB abort?

10.4 What is the difference between a latch and a lock? Give one thing each protects.

10.5 In crabbing, when is a child "safe" for an insert? For a delete?

Answers are in appendix B.

Further reading

Gray et al. [20]; Eswaran et al. [21]; Lehman and Yao [7]; Bayer and Schkolnick [8]; Petrov [1], chapters 5 and 6.

Part V

Recovery

Chapter 11

Logging and the Write-Ahead Rule

The buffer pool steals and does not force (chapter 3). So at the moment of a crash, the disk can hold changes from transactions that never committed, and can lack changes from transactions that did. The *log* is what lets the engine repair both. This chapter explains what goes in it, and the one rule that makes it trustworthy.

11.1 The failure model

Transaction failure. A transaction aborts: a deadlock victim, a constraint violation, or the user's choice. Its effects must be removed.

System failure. The process or machine stops. Memory is lost; the disk survives. Committed work must reappear; uncommitted work must vanish.

Media failure. The disk is lost. Only a backup or a replica helps; it is outside the scope of the log alone.

The log handles the first two. It is an append-only file of *log records*, each describing one change, written sequentially, which is the access pattern storage handles best.

11.2 Undo and redo

An update record carries enough to apply the change again (*redo*) and to reverse it (*undo*). For a page-oriented engine that might be the bytes before and after; for a triple store it can be logical: "insert triple $(5, 7, 9)$", whose undo is "delete $(5, 7, 9)$".

Every record has a *log sequence number* (LSN), increasing, unique, and often simply the record's byte offset in the log. Records of one transaction are chained by a *prevLSN* field, so the engine can walk a transaction's history backward when it must undo it.

Every page stores a *pageLSN*: the LSN of the last logged change applied to it. The pageLSN is how recovery knows whether a logged change already reached the page on disk: if pageLSN $\geq$ the record's LSN, it did.

11.3 The write-ahead rule

Key idea

Write-ahead logging (WAL). (1) A page may be written to disk only after every log record describing a change to it is durable: flush the log up to the page's pageLSN first. (2) A transaction commits only when its commit record, and therefore all its earlier records, are durable.

Rule 1 makes steal safe: if an uncommitted change reaches the disk, its undo information is already in the log. Rule 2 makes no-force safe: if a committed change has not reached the disk, its redo information is already in the log. Between them, every possible state of the disk after a crash can be repaired.

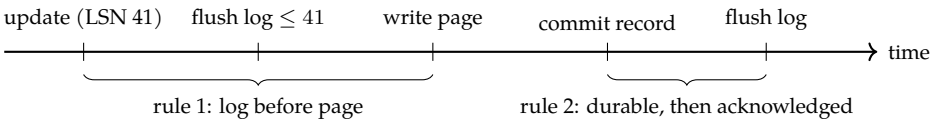


Figure 11.1: The two orderings the write-ahead rule imposes.

11.4 Making commits cheap: group commit

Rule 2 means every commit waits for an `fsync` of the log. If each commit issued its own `fsync`, throughput would be bounded by the device's `fsync` rate. *Group commit* lets commits that arrive while a flush is in progress share the next one:

```
function Commit(txn):
  lsn = Append(commit record for txn)       // into the in-memory log buffer
  wait until persistentLSN >= lsn           // a background flusher wakes us

flusher loop:
  wait until someone asked, or a short interval passes
  batch = swap out the log buffer
  write(batch); fsync()                     // one fsync for many commits
  persistentLSN = last LSN in batch; wake all waiters
```

Under concurrency, the number of `fsync` calls grows much more slowly than the number of commits. A single client still pays the full latency of one flush.

11.5 When a record is damaged

A crash can interrupt the last write to the log, leaving a *torn* record: fewer bytes than its header promises. Bit rot or a misbehaving device can damage a record whose bytes are all present. Robust logs detect both with two fields in every record header: its **length**, so a short record is recognized, and a **checksum** over its bytes, so a damaged one is. PostgreSQL, RocksDB, and

InnoDB all put a CRC on every log record. A recovery procedure then has a policy choice, discussed in chapter 13: stop at the first bad record and keep everything before it, or refuse to start.

In OntoDB: log records and the log manager

Every OntoDB log record has this layout (native byte order):

```
u32 size       whole record, header included
u32 crc32c     over every byte after this field
u8  type       begin | commit | abort | update | clr | checkpoint | term
i64 lsn, i64 txn_id, i64 prev_lsn
then, by type:
  update       i32 page_id, u8 is_insert, u64 subject, u64 predicate, u64 object
  clr          the update fields, then i64 undo_next
  checkpoint   u32 n, then n active transaction ids
  term         u64 term_id, u32 n, then n bytes of canonical spelling
```

The checksum is CRC-32C, the same checksum PostgreSQL, RocksDB, and InnoDB use. A short buffer raises an ordinary storage error (a torn write), while a record whose bytes are present but wrong raises a distinct corruption error, so the two cases can be handled differently. LSNs start at 1; reopening a log continues after its last valid record.
The log manager appends records to an in-memory buffer and a background thread performs group commit; a test runs four threads committing twenty times each and requires fewer `fsync`s than commits. A failed `fsync` is reported, never retried, for the reason in chapter 2.

Design decision 63: Length and CRC-32C in every record

Choice. Each record starts with its size and a CRC-32C over the rest. A short record raises a storage error (a torn write); a full-length record with a wrong checksum raises a distinct corruption error.

Alternatives. No checksum; a plain CRC-32; a faster non-cryptographic hash such as xxHash.

Why. A crash leaves a torn tail, and the two cases deserve different handling. CRC-32C has hardware instructions on x86 and ARM and is what PostgreSQL, RocksDB, and InnoDB use.

Cost. Eight bytes of header per record, and a checksum per append and per read.

Design decision 64: LSNs are counters, not byte offsets

Choice. An LSN is a 64-bit counter starting at 1. On open, the next LSN is one past the larger of the snapshot's LSN and the last valid record.

Alternatives. Use the record's byte offset in the log, as ARIES and PostgreSQL do, so an LSN locates its record directly.

Why. A checkpoint of the memory database empties the log file. A counter keeps increasing across that truncation, so "skip records at or below the snapshot's LSN" stays correct; an offset would restart at zero.

Cost. Finding a record by LSN needs a scan, not a seek.

Design decision 65: A failed `fsync` is fatal

Choice. When a log flush fails, the error is reported and the commit is not acknowledged. The flush is never retried.

Alternatives. Retry until it succeeds.

Why. After a failed `fsync`, the kernel may have dropped the dirty pages and cleared the error, so a retry that succeeds can still have lost the data (chapter 2).

Cost. A transient device error stops the database instead of being ridden out.

Status

Log records (plan 8.1) and the log manager with group commit (plan 8.2) run today. The buffer pool already applies rule 1 when it writes a page back: it flushes the log up to the page's pageLSN first, with a crash point between the two. Plan 8.3 completes the rule by making every update set the pageLSN.

Check your understanding

11.1 State the two write-ahead rules, and say which buffer-pool policy each one makes safe.

11.2 A page has pageLSN 40. Recovery reads an update record for it with LSN 35. Redo it or skip it?

11.3 Ten sessions commit within one millisecond. Without group commit, how many `fsync`s? With it, at least how many?

11.4 A log's last record has a valid header claiming 60 bytes, and only 41 are in the file. Torn or corrupt? What if all 60 are there but the checksum fails?

11.5 Why does OntoDB number LSNs with a counter rather than a byte offset?

Answers are in appendix B.

Further reading

Mohan et al. [2], sections 1–3; Gray and Reuter [3], chapters 9–10; Petrov [1], chapter 5.

Chapter 12

ARIES

ARIES (Algorithm for Recovery and Isolation Exploiting Semantics), published by Mohan, Haderle, Lindsay, Pirahesh, and Schwarz in 1992 [2], is the recovery algorithm behind most disk-based database systems. It supports steal and no force, fine-grained locking, and partial rollbacks, and it recovers correctly even if the system crashes again *during* recovery. Its central idea fits in three words: **repeat history, then undo**.

12.1 The data structures

Log records as in chapter 11: LSN, transaction id, prevLSN, and type-specific fields.

pageLSN on every page: the LSN of the last change applied to it.

Transaction table for every active transaction: its status and *lastLSN*, the LSN of its most recent record.

Dirty page table for every page dirty in the buffer pool: its *recLSN*, the LSN of the first record that made it dirty since it was last read from disk. Any change to the page before recLSN is already on disk.

Compensation log records (CLRs) written when a change is undone. A CLR describes the undo, and carries *undoNextLSN*: the next record of that transaction still to undo.

12.2 Checkpoints

Without checkpoints, recovery would read the log from the beginning of time. A *fuzzy checkpoint* writes the transaction table and the dirty page table into the log without stopping the system or forcing any pages, and records the checkpoint's LSN in a well-known place. Recovery then starts from the last checkpoint: the tables in it tell analysis everything that happened before.

12.3 Pass 1: Analysis

Scan the log forward from the last checkpoint, rebuilding the two tables as of the crash:

- On any record of a transaction not in the table, add it. Update its lastLSN.
- On a commit, mark it committed; on an end record, remove it.
- On an update or CLR for a page not in the dirty page table, add the page with recLSN = this LSN.

At the end, every transaction still in the table without a commit is a *loser*. The *redoLSN* is the smallest recLSN in the dirty page table: no earlier change can be missing from disk.

12.4 Pass 2: Redo

Scan forward from redoLSN, and reapply every update and every CLR, for winners and losers alike, unless the change is provably already on disk:

```
for each update or CLR record r, from redoLSN forward:
  if r.page not in dirty page table:          skip   // page was clean at the crash
  if dirtyPageTable[r.page].recLSN > r.lsn:   skip   // flushed after r
  fetch page
  if page.pageLSN >= r.lsn:                   skip   // r already on the page
  apply r to the page; page.pageLSN = r.lsn
```

Redo **repeats history**: it brings the database to exactly its state at the crash, uncommitted changes included. That sounds wasteful, but it makes undo simple: undo now starts from a precise, known state, and can use the same logical operations that a normal abort uses.

12.5 Pass 3: Undo

Undo the losers, newest change first, across all of them at once:

```
toUndo = { lastLSN of each loser }
while toUndo is not empty:
  r = record with the largest LSN in toUndo; remove it
  if r is a CLR:
    if r.undoNextLSN exists: add r.undoNextLSN to toUndo
    else: write an end record for r.txn
  else if r is an update:
    undo r; write CLR(undoNextLSN = r.prevLSN)
    if r.prevLSN exists: add r.prevLSN to toUndo
    else: write an end record for r.txn
```

Key idea

A CLR is redone but never undone. If the system crashes during undo, the next recovery redoes the CLRs (history repeats), then follows their undoNextLSN pointers past everything already undone. Recovery work is therefore bounded no matter how many times the system crashes during recovery.

12.6 A worked example

Two transactions run. T_1 commits; T_2 does not. The buffer pool happened to write page P2 (stealing T_2's uncommitted change), but never wrote P1.

LSN	Transaction	Record	prevLSN
10	T_1	update P1	—
20	T_2	update P2	—
30	T_1	commit	10
40	T_2	update P1	20

crash. On disk: P1 has pageLSN 0; P2 has pageLSN 20.

Analysis. T_1 committed; T_2 is a loser with lastLSN 40. The dirty page table is {P1: recLSN 10, P2: recLSN 20}, so redoLSN is 10.

Redo. LSN 10 on P1: pageLSN $0 < 10$, so redo; P1's pageLSN becomes 10. LSN 20 on P2: pageLSN $20 \geq 20$, so skip; the stolen write already put it there. LSN 40 on P1: pageLSN $10 < 40$, so redo; pageLSN becomes 40. The database is now exactly as it was at the crash, including T_2's two changes.

Undo. toUndo = {40}. Undo LSN 40 on P1, write CLR 50 with undoNextLSN 20. Undo LSN 20 on P2, write CLR 60 with no undoNextLSN, then an end record for T_2. T_1's change survives; T_2 left no trace.

If the system had crashed right after CLR 50, the next analysis would find T_2 with lastLSN 50; redo would reapply CLR 50; undo would read CLR 50, jump straight to LSN 20, and never undo LSN 40 a second time.

12.7 Structure changes: nested top actions

A B+ tree split moves keys between pages. Suppose T inserts a key, which splits a leaf, and later aborts. Undoing "the split" would be wrong: another transaction may have committed an insert into the new leaf since then. The split is a change to the *structure*, which is valid regardless of who caused it; only the *logical* insert belongs to T.

ARIES logs a split as a *nested top action*: the split's page records, followed by a dummy CLR whose undoNextLSN points to the record just before the split. Redo replays the split like any other change. Undo, following the dummy CLR, skips over it. The logical "insert key k" is logged separately, and its undo is the logical "delete key k", performed wherever that key now lives, which is why undo in ARIES is logical while redo is physical.

In OntoDB: ARIES

OntoDB's page-based recovery follows this chapter: steal and no force, page-oriented redo, logical undo of triples, CLRs that are redone and never undone, and B+ tree splits logged as redo-only nested top actions with the triple insert logged separately. On open, recovery prints one line per pass (`analysis`, `redo`, `undo`). The bar for the feature is a crash harness that kills the process at random points (including named crash points inside splits and between a log flush and a page write) and compares the reopened store with an oracle that replayed only acknowledged commits: `scripts/crashtest -seeds 200` must pass.

Design decision 66: A blocking checkpoint first (planned)

Choice. The page store's `\checkpoint` may pause updates while it writes the checkpoint record; the comment above `Checkpoint` names which kind it is.

Alternatives. ARIES's fuzzy checkpoint, which never stops the system.

Why. A blocking checkpoint is easy to reason about and enough to bound recovery. Fuzzy checkpointing is an optimization to add once the three passes are correct.

Cost. Updates stall for the length of the checkpoint.

Design decision 67: Recovery is tested against an oracle

Choice. A crash harness kills the process at random points and at named crash points, reopens the store, and compares it with an oracle that applied only acknowledged commits. The harness is itself tested on deliberately broken stores.

Alternatives. Hand-written recovery scenarios only.

Why. The bugs in recovery live in interleavings nobody thinks to write. Many random crash points find them; a harness that catches a planted lost commit proves it can.

Cost. Slow tests, and failures that need a seed to reproduce.

Status

The log record types used here, including CLRs and checkpoints, exist and serialize (plan 8.1). Abort by the log, checkpoints, the three passes, and the crash test are plans 8.4 to 8.7, after the buffer pool and B+ tree. The crash harness already detects a lost commit and a surviving uncommitted write on deliberately faulty test stores.

Check your understanding

12.1 In the worked example, suppose the pool had also written P1 after LSN 40. What does redo do now, and what does undo do?

12.2 Why does ARIES redo the losers' changes, only to undo them right after?

12.3 The system crashes after CLR 60 but before T_2's end record. What does the next recovery do?

12.4 Why is a B+ tree split never undone, even when the transaction that caused it aborts?

12.5 What do the dirty page table's recLSNs let redo skip?

Answers are in appendix B.

Further reading

Mohan et al. [2], the whole paper; Ramakrishnan and Gehrke [4], chapter 18, for a gentler walkthrough; Petrov [1], chapter 5.

Chapter 13

Durability in Practice

ARIES is the general answer for a page-based engine. A simpler engine can make a simpler choice and still be correct. OntoDB's persistent databases, which run today, are such a design: an in-memory database made durable by a **snapshot** plus a **logical, redo-only write-ahead log**. This chapter explains the design through the ideas of the previous chapters, because every choice in it is one of those ideas in a smaller form.

13.1 Where it sits in the steal/force table

The database lives entirely in memory; the files are its durable copy. Nothing is ever written to the data files except a whole snapshot, and a snapshot is taken only between statements, when every change in memory is committed. In the terms of table 3.2:

- **No steal.** Uncommitted changes never reach the snapshot. So recovery never needs to *undo* anything on disk.
- **No force.** A commit does not rewrite the snapshot; it appends to the log. So recovery must *redo* committed changes made since the last snapshot.

No steal plus no force is the *redo-only* cell: the log needs redo information and nothing else, and recovery is a single forward pass.

Key idea

Choosing a cell of the steal/force table chooses the recovery algorithm. Steal and no force need undo and redo (ARIES). No steal and no force need only redo. The price of no steal is that all uncommitted state must fit in memory, which an in-memory database accepts by definition.

13.2 Logical logging

Page-based ARIES logs *physical* or *physiological* changes: this page, these bytes, or this page, this slot. The memory database has no pages, so it logs *logical* operations: "insert triple (18, 9, 4)". Logical records are small and independent of storage layout, and they are safe here only because there is no partially written page that could make a logical operation ambiguous on replay.

Ids in those records must mean the same terms after a restart, so the log also carries the dictionary: a *term record* for each spelling the first time a transaction uses it.

13.3 One statement, one transaction

1. The statement runs against a *recording store*, a decorator around the real store that remembers every triple it actually changed.
2. The log receives **begin**, a **term** record for each new spelling, one **update** per changed triple, and **commit**.
3. The commit is forced (chapter 11) before the shell prints the result.

If the statement fails part way, the recording store's changes are undone in memory, in reverse order, and nothing it needs reaches the log. A statement that changed nothing logs nothing.

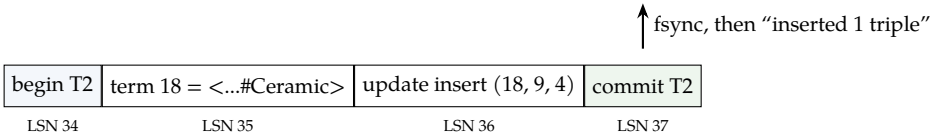


Figure 13.1: The log records of one `INSERT DATA` in a persistent database.

13.4 Recovery

Opening a database directory:

1. Load the snapshot, if there is one: every term in id order, then every triple.
2. Scan the log and stop at the first record that is torn or fails its checksum (chapter 11).
3. Skip records at or below the snapshot's LSN; the snapshot already contains them.
4. Apply *every* term record in order, whatever its transaction did: the dictionary only grows, and a term no committed triple uses is invisible.
5. Apply update records only for transactions that have a commit record, in log order.

Step 5 is the whole of "undo" in a redo-only design: an uncommitted transaction's records are simply never applied.

13.5 Checkpoints as atomic snapshots

A checkpoint writes the full state to `snapshot.tmp`, `fsyncs` it, `renames` it over `snapshot`, `fsyncs` the directory, and then empties the log. Each step is safe to crash after:

Crash after	On reopen
writing `snapshot.tmp`	old snapshot, full log: nothing lost
the rename	new snapshot, old log: records at or below its LSN are skipped
emptying the log	new snapshot, empty log

This is the log-structured idea of chapter 6 with a single level: an append-only log, periodically compacted into one sorted image.

13.6 Damaged logs: recovery modes

When the scan in step 2 stops early, there are two defensible policies, named after RocksDB's WAL recovery modes:

Point in time (the default). Keep every transaction before the bad record, truncate the log there, and report how many bytes were discarded. A torn final record is the normal shape of a crash, and this is the right answer for it; a transaction whose commit is after the damage is treated as uncommitted, exactly as a crash would.

Absolute consistency (`-strict`). Refuse to open, so an operator can inspect the files before anything is truncated.

A damaged snapshot always refuses to open: it has no valid prefix to fall back to.

13.7 Verification

Detecting damage early is the other half of durability. The `\verify` command follows the tradition of PostgreSQL's `pg_amcheck`, MySQL's `CHECK TABLE`, and RocksDB's `VerifyChecksum`: it changes nothing and reports four checks, one line each.

```
snapshot: ok (lsn 37, 18 terms, 15 triples)
wal: ok (0 records, 0 bytes)
replay: matches the live database (15 triples)
invariants: ok (no duplicate triples, every id in the dictionary)
verify: ok
```

The replay check is the strongest: it rebuilds the database from the two files into a fresh store and compares it with the one in memory. If they differ, the files would not reproduce what the user sees.

13.8 Testing a recovery design

A recovery path that is never exercised does not work. OntoDB tests this one with *crash points*: named places in the code, such as just before a commit record is appended and just after a snapshot is renamed, where a test can make the operation stop as if the process died. Scenario tests then reopen the directory and check what survived: a crash before commit leaves no trace, a crash after the snapshot rename does not replay records twice, a torn tail keeps the committed prefix, a flipped byte in the middle of the log drops everything after it, and a tampered log fails verification.

In OntoDB: what runs

All of this chapter runs today: start the shell as `ontodb mydb`. `\load` and every update are durable transactions, `\checkpoint` writes a snapshot, `\log` prints the decoded log, `\verify` checks the files, and `\crash` exits without flushing, to try recovery by hand. Seventeen scenario tests cover the cases above, and they pass under AddressSanitizer, UndefinedBehaviorSanitizer, and ThreadSanitizer.
When the indexed store arrives, the same log records, log manager, and group commit carry over; what changes is that pages can be stolen, so recovery gains undo, pageLSNs, and the three passes of chapter 12.

Design decision 68: Terms replay whatever their transaction did

Choice. On recovery, every term record is applied in order, even if its transaction never committed; update records apply only for committed transactions.

Alternatives. Replay terms only for committed transactions.

Why. Ids are dense, and each term record carries the id it was given. If an aborted transaction's term were skipped, every later term would land on a different id than the log recorded. A term no committed triple uses is invisible.

Cost. Aborted transactions can leave unused terms behind.

Design decision 69: Point-in-time recovery by default

Choice. A damaged log is truncated at the first bad record, and the database opens with everything before it, reporting what was dropped. `-strict` refuses to open instead.

Alternatives. Strict by default; or skip the bad record and continue.

Why. A torn final record is the ordinary result of a crash, and the right response is to open. Skipping a bad record would apply later transactions on top of a missing one.

Cost. Damage in the middle of the log silently loses every later commit apart from the report, which is why `\verify` exists.

Check your understanding

13.1 Why can the memory database's log omit undo information?

13.2 A crash happens after the snapshot rename but before the log is emptied. Which records does reopen apply?

13.3 A transaction logs a term record for id 19 and then fails. A later committed transaction logs id 20. Why must replay still apply the term for 19?

13.4 When would you choose `-strict` over point-in-time recovery?

13.5 Why is the replay check of `\verify` stronger than checking checksums alone?

Answers are in appendix B.

Further reading

Gray and Reuter [3] on redo-only logging; DeWitt et al. [24] on recovery for main-memory databases; the RocksDB WAL recovery-mode documentation [25].

Chapter A

A Map from Chapters to OntoDB

Each chapter's ideas, the OntoDB plan items that implement them, and where the code lives. Plan items are listed in the repository's `PLAN.md`; run `scripts/check <feature>` to build and test one.

Chapter	Plan items	Where
1 The engine	0.1, 0.3	`tools/shell`, `src/common/database.cpp`
2 Pages	1.1, 3.1	`src/storage/disk`, `src/storage/page`, `src/storage/term`
3 Buffer pool	1.2–1.6	`src/buffer`, `src/storage/page/page_guard.cpp`
4 B+ trees	2.1–2.7, 6.1–6.2	`src/storage/index/b_plus_tree.cpp`
5 Indexing RDF	0.2, 3.2–3.6	`src/storage/index/triple_key.cpp`, `src/dictionary`, `src/catalog`, `src/store`
6 LSM trees	—	(contrast only)
7 Plans	4.4–4.6	`src/parser`, `src/binder`, `src/planner`, `src/execution`
8 Joins	4.1–4.3, 5.1–5.5	`src/execution/executors`, `src/execution/row_merge.cpp`, `src/optimizer`, `src/stats`
9 Transactions	7.2, 7.4	`src/concurrency`, `test/isolation/specs`
10 Concurrency	7.1–7.6	`src/concurrency`, B+ tree crabbing
11 Logging	8.1–8.3	`src/recovery/log_record.cpp`, `src/recovery/log_manager.cpp`
12 ARIES	8.4–8.7	`src/recovery`, `test/recovery`, `scripts/crashtest`
13 Durability	9.1–9.7	`src/recovery/durability.cpp`, `src/recovery/snapshot.cpp`, `test/durable`

The OntoDB manual (`https://lamng3.github.io/ontodb-docs/`) is the companion to this book: it documents how to use each feature, while this book explains why it is built the way it is.

Chapter B

Answers

Short answers to the questions at the end of each chapter. Try a question before reading its answer; most of them take one line of reasoning, and that line is the point.

chapter 1: What a Database Engine Is

1.1 (a) The B+ tree, an access method. (b) The buffer pool. (c) The optimizer.

1.2 It touches every layer: executors take locks on the keys they read and write, every page change is logged, and commit is decided at the interface. It cuts across the stack instead of sitting in it.

1.3 A five-way self-join of the triple table on the subject column, each pattern being a selection on that table, followed by a projection.

1.4 Only `Metal`. The stored triples say `Metal subClassOf Material`; `Alloy subClassOf Material` would have to be inferred, and OntoDB has no reasoner.

chapter 2: Disks, Files, and Pages

2.1 At byte $37 \times 4096 = 151{,}552$. Pages are fixed-size, so the offset is arithmetic on the page id.

2.2 The index points at the record id, (*page, slot*). Compaction changes the offset stored in the slot, not the slot number.

2.3 No: it was only in the operating system's page cache. `fsync` on the file (and, for a new file, on its directory) makes it durable.

2.4 The kernel may already have dropped the dirty pages and cleared the error, so the second `fsync` "succeeds" with the data gone. The only safe response is to stop and recover from the log.

2.5 Zero is what unwritten memory and files contain. With -1 as the sentinel, a forgotten initialization shows up as an error instead of as a tree rooted at page 0, which is a legal page.

2.6 OntoDB refuses it: a term heap record must fit on one page (at most 4,076 bytes), so the term cannot be interned and the load fails as one transaction. PostgreSQL would compress it and, if still large, move it out of line into a TOAST table, leaving a pointer in the row.

2.7 Allocation grew the file, but the new length was never made durable (no `fsync` before the crash), while the log record that uses the page was forced. Redo reads the page past the end as zeros and reapplies the record; writing the page back grows the file.

chapter 3: The Buffer Pool

3.1 It must be written back to disk (after the log is flushed up to its pageLSN, by the write-ahead rule) before the frame is reused.

3.2 At $t = 11$ the backward 2-distances are A: $11 - 8 = 3$, B: $11 - 3 = 8$, C: $11 - 7 = 4$. LRU-K evicts B.

3.3 X. Both have fewer than three accesses, so both have infinite distance, and the tie goes to the earliest oldest access: X's is at time 2, Y's at time 5. Recency does not enter until a frame has K accesses.

3.4 A scan touches each page once; under LRU every scanned page becomes the most recent, so the scan pushes out the hot pages. With $K = 2$, a page touched once has infinite distance and leaves first. $K = 1$ is LRU.

3.5 Both steal cells. Under steal, an uncommitted change can reach the disk, and the log must be able to take it back.

3.6 The bytes just written: one thread runs requests in the order scheduled. With several I/O threads the read could run first and return the old page, unless requests for the same page were ordered some other way.

3.7 Until the first thread's read finishes: the pool's one latch is held across the I/O, and the single I/O thread would serialize the reads anyway.

3.8 It would write the very page the crash point was meant to keep off the disk, so the test could not observe the state a real crash leaves.

3.9 Once the pin is gone the pool may evict the frame and load a different page into it; a latch still held would then be a latch on someone else's page.

3.10 Page 7 is now resident (another page may have been evicted to make room), and its LRU-K history holds two recent accesses, so it has a finite, small distance and will be among the last to go.

3.11 That frame can never be evicted, so the pool is permanently one frame smaller; enough leaks and every frame is pinned and the next miss fails. Guards unpin exactly once at the end of their scope, and the tests check that nothing stays pinned.

chapter 4: B-Trees and B+ Trees

4.1 A 24-byte key plus a 4-byte child page id. n keys separate $n + 1$ children, so the leftmost child has no key of its own, and storing slots uniformly simply leaves that key unused.

4.2 $300^3 = 2.7 \times 10^7 < 10^8 \leq 300^4$, so four levels. The root and the roughly 300 pages below it are always cached, so a lookup costs about two reads.

4.3 The right (new) leaf, whose first key it is. Left in the left leaf, a key equal to the separator sits in the left child, which breaks the separator rule, and a search for 14 goes right.

4.4 Leaf keys are the data, and every key must stay in a leaf. Internal keys only route searches, so the middle one can move up and leave both halves.

4.5 Holding two leaves, and possibly a parent, can use up a small pool; with latches, two scans moving in opposite directions can deadlock.

4.6 A split would have to rewrite the parent pointer in every child that moved, up to 72 pages for one internal split. Without them, a structure change re-fetches parents remembered by id, which is safe only while one thread changes the tree; latch crabbing fixes that.

4.7 Two leaves of 85. A leaf of 1 would be below the minimum of 84 keys, so the last two leaves share their 170 keys evenly.

4.8 A full leaf splits into two half-full leaves, so random inserts leave leaves about two-thirds full, and each insert is a random descent. Bulk load writes full leaves sequentially.

chapter 5: Indexing RDF

5.1 258 is `0x0102`. Big-endian: `00 00 00 00 00 00 01 02`; little-endian: `02 01 00 00 00 00 00 00`. Against 3 (`...00 03` and `03 00...`), big-endian puts 258 above 3, correctly; little-endian puts it below.

5.2 `(5, ?, ?)`: $[(5,0,0),(6,0,0))$. `(5, 7, ?)`: $[(5,7,0),(5,8,0))$.

5.3 With $n = 2$, the predicate overflows to 0 and the carry bumps the subject: $(6,0,0)$. With $n = 1$ and subject $2^{64}-1$, nothing is above the prefix, so the bound saturates to $(2^{64}-1, 2^{64}-1, 2^{64}-1)$.

5.4 Object bound: OSP. Subject and object bound: OSP, whose prefix is object then subject.

5.5 Otherwise one term gets two ids, and a join between loaded data and a query constant silently matches nothing. Example: a datatype written with a prefix, `"21"^^xsd:integer`, and the same datatype written as a full IRI must become one spelling.

5.6 Every operator would read that term as "unbound": a join would overwrite the slot from the other side, a scan would treat a constant as a variable, and the saturated upper bound could exclude a real key. The answers would be wrong without any error.

5.7 With SPO, POS, and OSP, every pattern shape is already a prefix of some index, so it is already a range scan; another order can only offer the same range sorted differently. That matters to an operator that wants its input in a particular order, such as a merge join on the object, or a sort that an index could avoid.

5.8 Range scans: subject bound; subject and predicate; all three; and none (the whole tree is the range). Subject and object bound is a subject range filtered by object. Predicate only, object only, and predicate with object are full scans with a filter.

5.9 The prefix scan on the hash returns both ids; the dictionary reads each candidate's spelling from the heap and returns the one that is equal. A collision costs an extra read, never a wrong answer.

5.10 A root moves whenever it splits or collapses; the header page absorbs those changes. The catalog changes only when an index is created or dropped.

chapter 6: The Log-Structured Alternative

6.1 An LSM tree: writes are sequential and batched. A B+ tree pays a random page write for each insert; the LSM tree pays later in compaction, and in read amplification.

6.2 Older versions of the key sit in immutable SSTables that are never edited. The tombstone shadows them until compaction drops both.

6.3 One wasted SSTable read. A Bloom filter never says "absent" for a present key.

6.4 Appending to the log is the write path, and a checkpoint rewrites the whole state as one sorted image, which is a compaction into a single level that discards the old log.

chapter 7: From SPARQL to a Plan

7.1 `?s` is slot 0, `?name` slot 1, `?age` slot 2, in order of first appearance.

7.2 Rows can differ in slots that are not projected: two students with the same name are two scan rows but one projected row.

7.3 Ids are assigned in load order, not value order; `"10"` can have a smaller id than `"9"`. Sorting must compare values.

7.4 Volcano stops as soon as ten rows reach the root, doing only the join work that produced them. Materializing computes the entire join first, then keeps ten rows.

7.5 Run generation writes 100 runs of 10 pages (1,000 pages). Merging 8 at a time gives 13 runs, then 2, and 2 is few enough for the final merge to stream to the consumer: two intermediate passes, each writing 1,000 pages. In all, 3,000 pages written (and 3,000 read back), plus the input scan.

7.6 Neither. Without `ORDER BY` the order of solutions is unspecified, so any two of the matching names are a correct answer. The backends produce different orders (insertion order against key order).

7.7 An empty pattern has exactly one solution, the empty binding, and that row is the identity for a join. Zero rows would make every join empty.

chapter 8: Joins and Query Optimization

8.1 Nested loop: $10^3 \times 10^6 = 10^9$ comparisons. Index nested loop: 1,000 prefix probes, each a root-to-leaf descent (mostly cached) plus the few matching triples.

8.2 When both inputs are large, no index makes a probe selective, and the build side fits in memory.

8.3 The exact number of triples with that predicate and object, read from the pair counts. Errors return at joins, where the estimator must assume how join values are distributed, and after updates, until the statistics are collected again.

8.4 $(10^4 \times 10^3)/10^6 = 10$. The q-error is $\max(10/1000,\ 1000/10) = 100$.

8.5 It keeps the 2,000 course rows, keyed by `?c`, and makes 50,000 lookups, one per probe row. Swapped, it would keep 50,000 rows to answer 2,000 lookups: the same answer for 25 times the memory, which is why the build side should be the smaller one once estimates exist.

8.6 With rebinding, each of the 1,000 outer rows binds `?s`, so the inner scan is a prefix probe returning that student's names: about 1,000 probes reading a few triples each. Without it, every outer row rescans all 1,000,000 `ub:name` triples: 10^9 triples read.

8.7 A join's estimate is built from its inputs' estimates, so their errors multiply: two inputs each off by 10 give an output off by up to 100.

8.8 `\explain` prints the optimizer's plan. If the executor swapped algorithms at run time, the printed plan would not be the one that ran.

8.9 It joins three type patterns that share no variable: about 1,900 graduate students times 1,000 universities times 15 departments, tens of millions of rows of cross product, before the first pattern that connects them. Any operator must produce those rows; only putting a connected pattern first avoids them.

chapter 9: Transactions and Isolation

9.1 $r_1(A)$ before $w_2(A)$ gives $T_1 \to T_2$; $r_2(B)$ before $w_1(B)$ gives $T_2 \to T_1$. The graph has a cycle, so the schedule is not conflict-serializable.

9.2 $r_1(A)\, w_2(A)\, c_2\, r_1(A)$: T_1 reads two different values. Repeatable read is the first level that forbids it.

9.3 Validity is defined by the application's invariants, such as "every student has a name". The engine cannot know them; it provides atomicity, isolation, and durability, so that the application can keep them.

9.4 Nothing from the statement: the insert operator deletes the two triples it had added before the error propagates. That holds even with no log.

9.5 Both read N and both insert a student they believe is number $N + 1$: an anomaly built on a phantom (write skew). It is possible at read uncommitted, read committed, and repeatable read. Serializable's range locks make one insert wait; if both hold the range in S and both want to insert, it is a deadlock, and the younger transaction is aborted.

chapter 10: Concurrency Control

10.1 Plain 2PL may release an X lock in its shrinking phase, before commit; another transaction reads the value and must abort if the writer later aborts. Strict 2PL holds X locks until commit or abort, so no one reads an uncommitted write.

10.2 IS on the database and S on the `"spo"` index, with no key locks. Any writer below must hold IX on the index, and IX conflicts with S, so the one check on the index covers every key.

10.3 T_5, the younger (larger id).

10.4 A latch is a short mutex on a physical page for one operation; it protects a page's structure during a split. A lock is held for a transaction on a logical item, such as a triple key, and provides isolation.

10.5 For an insert, a child is safe if it is not full, so it cannot split. For a delete, it is safe if it is more than half full, so it cannot underflow.

chapter 11: Logging and the Write-Ahead Rule

11.1 Rule 1: a page is written only after the log records for its changes are durable; this makes steal safe. Rule 2: a commit is acknowledged only after its commit record is durable; this makes no force safe.

11.2 Skip it: pageLSN $40 \geq 35$, so the change is already on the page.

11.3 Without group commit, ten. With it, as few as one, or two if a flush was already running when they arrived.

11.4 Short: a torn write. Full length with a bad checksum: corruption. In point-in-time mode, either one ends the valid prefix there.

11.5 A checkpoint of the memory database empties the log file. Byte offsets would restart at zero; a counter keeps increasing, so comparisons with the snapshot's LSN stay correct.

chapter 12: ARIES

12.1 Analysis is unchanged: the dirty page table still has P1 at recLSN 10, because nothing logged the flush. Redo fetches P1, sees pageLSN $40 \geq$ each record's LSN, and skips all three. Undo is unchanged: undo 40 (CLR 50), undo 20 (CLR 60), end.

12.2 Repeating history puts the database in exactly its state at the crash, so undo can use the same logical operations as a normal abort, and CLRs from an interrupted earlier undo are replayed, not repeated.

12.3 Analysis finds T_2 a loser with lastLSN 60. Redo reapplies CLRs 50 and 60 if the pages lack them. Undo reads CLR 60, which has no undoNextLSN, and writes the end record. Nothing is undone twice.

12.4 The split is a structural change, valid whoever caused it, and other transactions may since have inserted into the new leaf. Only the logical insert belongs to the transaction, and its undo is a logical delete.

12.5 Records for pages that were not dirty at the crash, and records older than a page's recLSN, without reading the page.

chapter 13: Durability in Practice

13.1 No steal: uncommitted changes never reach the snapshot, and replay simply does not apply an uncommitted transaction's updates.

13.2 The new snapshot. Every record in the old log is at or below the snapshot's LSN, so none is applied.

13.3 Ids are dense, and each term record carries its id. Skipping 19 would put the next term on id 19, so committed triples naming 20 would mean a different term; OntoDB's replay detects this and refuses.

13.4 When damage could come from something other than a crash (a failing disk, copied or edited files) and nothing should be truncated before someone looks.

13.5 Checksums show each record is intact as written. Replay shows the files actually reproduce the live database, which also catches logic bugs, such as a change that was never logged, that leave perfectly valid checksums.

Bibliography

[1] A. Petrov. *Database Internals: A Deep Dive into How Distributed Data Systems Work*. O'Reilly, 2019.

[2] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, and P. Schwarz. ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992.

[3] J. Gray and A. Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1993.

[4] R. Ramakrishnan and J. Gehrke. *Database Management Systems*, 3rd edition. McGraw-Hill, 2003.

[5] R. Bayer and E. McCreight. Organization and maintenance of large ordered indexes. *Acta Informatica*, 1(3):173–189, 1972.

[6] D. Comer. The ubiquitous B-tree. *ACM Computing Surveys*, 11(2):121–137, 1979.

[7] P. L. Lehman and S. B. Yao. Efficient locking for concurrent operations on B-trees. *ACM Transactions on Database Systems*, 6(4):650–670, 1981.

[8] R. Bayer and M. Schkolnick. Concurrency of operations on B-trees. *Acta Informatica*, 9(1):1–21, 1977.

[9] E. J. O'Neil, P. E. O'Neil, and G. Weikum. The LRU-K page replacement algorithm for database disk buffering. In *SIGMOD*, 1993.

[10] W. Effelsberg and T. Härder. Principles of database buffer management. *ACM Transactions on Database Systems*, 9(4):560–595, 1984.

[11] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil. The log-structured merge-tree (LSM-tree). *Acta Informatica*, 33(4):351–385, 1996.

[12] B. H. Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.

[13] C. Weiss, P. Karras, and A. Bernstein. Hexastore: Sextuple indexing for semantic web data management. *Proceedings of the VLDB Endowment*, 1(1):1008–1019, 2008.

[14] T. Neumann and G. Weikum. RDF-3X: A RISC-style engine for RDF. *Proceedings of the VLDB Endowment*, 1(1):647–659, 2008.

[15] G. Graefe. Query evaluation techniques for large databases. *ACM Computing Surveys*, 25(2):73–169, 1993.

[16] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. Access path selection in a relational database management system. In *SIGMOD*, 1979.

[17] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann. How good are query optimizers, really? *Proceedings of the VLDB Endowment*, 9(3):204–215, 2015.

[18] S. Harris and A. Seaborne, editors. SPARQL 1.1 Query Language. W3C Recommendation, 2013.

[19] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil. A critique of ANSI SQL isolation levels. In *SIGMOD*, 1995.

[20] J. Gray, R. A. Lorie, G. R. Putzolu, and I. L. Traiger. Granularity of locks and degrees of consistency in a shared data base. In *Modelling in Data Base Management Systems*, 1976.

[21] K. P. Eswaran, J. N. Gray, R. A. Lorie, and I. L. Traiger. The notions of consistency and predicate locks in a database system. *Communications of the ACM*, 19(11):624–633, 1976.

[22] T. S. Pillai, V. Chidambaram, R. Alagappan, S. Al-Kiswany, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau. All file systems are not created equal: On the complexity of crafting crash-consistent applications. In *OSDI*, 2014.

[23] PostgreSQL Global Development Group. Fsync errors ("fsyncgate"). PostgreSQL wiki, 2018. `https://wiki.postgresql.org/wiki/Fsync_Errors`.

[24] D. J. DeWitt, R. H. Katz, F. Olken, L. D. Shapiro, M. R. Stonebraker, and D. Wood. Implementation techniques for main memory database systems. In *SIGMOD*, 1984.

[25] RocksDB. WAL recovery modes. RocksDB wiki. `https://github.com/facebook/rocksdb/wiki/WAL-Recovery-Modes`.